



US 20250285011A1

(19) **United States**

(12) **Patent Application Publication**
Smith et al.

(10) **Pub. No.: US 2025/0285011 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **SYSTEM AND METHODS FOR AN
ADAPTIVE MACHINE LEARNING MODEL
SELECTION BASED ON DATA
COMPLEXITY AND USER GOALS**

(71) Applicant: **The Strategic Coach Inc.**, Toronto
(CA)

(72) Inventors: **Barbara Sue Smith**, Toronto (CA);
Daniel J. Sullivan, Toronto (CA)

(73) Assignee: **The Strategic Coach Inc.**, Toronto, ON
(CA)

(21) Appl. No.: **18/754,654**

(22) Filed: **Jun. 26, 2024**

Related U.S. Application Data

(63) Continuation-in-part of application No. 18/600,487,
filed on Mar. 8, 2024, now Pat. No. 12,086,721.

Publication Classification

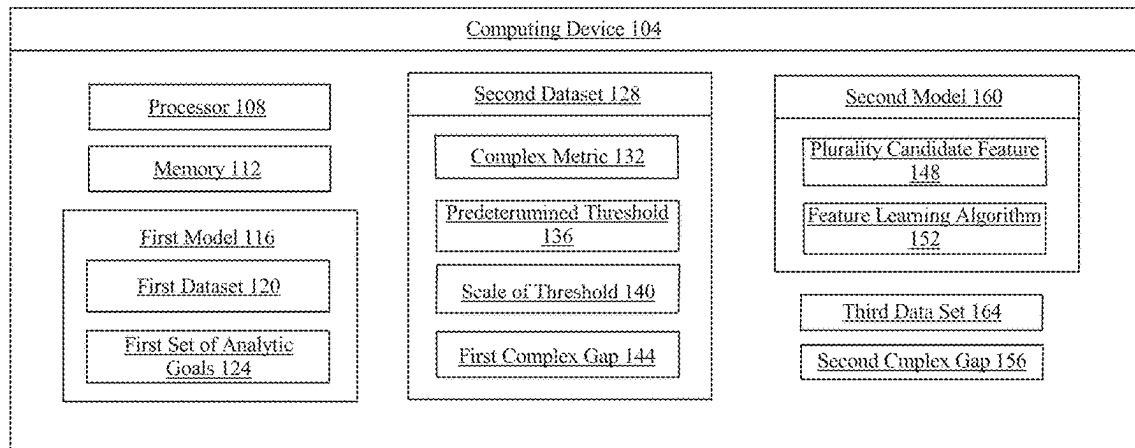
(51) **Int. Cl.**
G06N 20/00 (2019.01)

(52) **U.S. Cl.**
CPC **G06N 20/00** (2019.01)

(57) **ABSTRACT**

The apparatus employs adaptive machine learning for model selection based on data complexity and user goals. It consists of a processor and memory. Initially, it creates a first model from a dataset and analytic goals. Then, it determines a complexity metric for another dataset. Using a feature learning algorithm, it extracts candidate features from the second dataset. From these features, it generates a second model. The device assesses this model's performance using a third dataset and selects it based on its relation to the complexity gap.

100 →



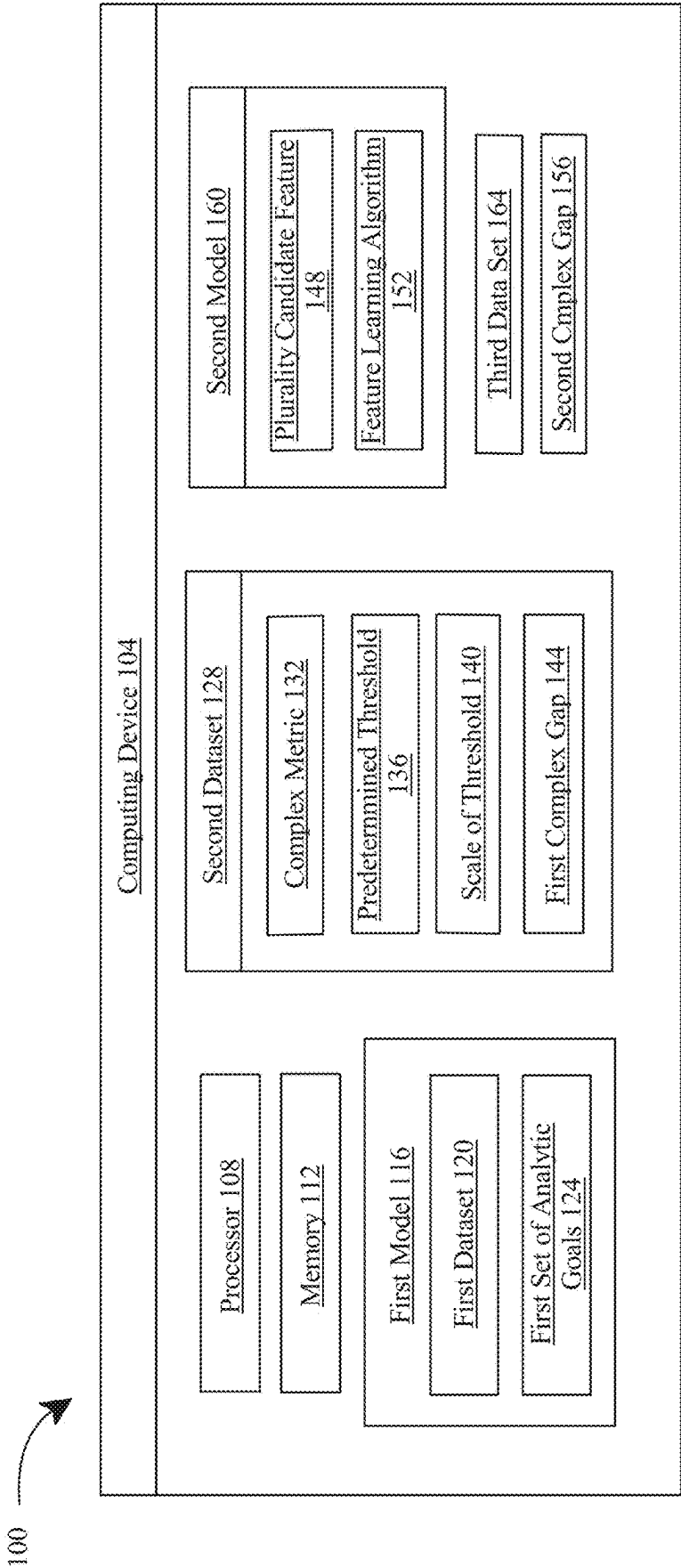


FIG. 1

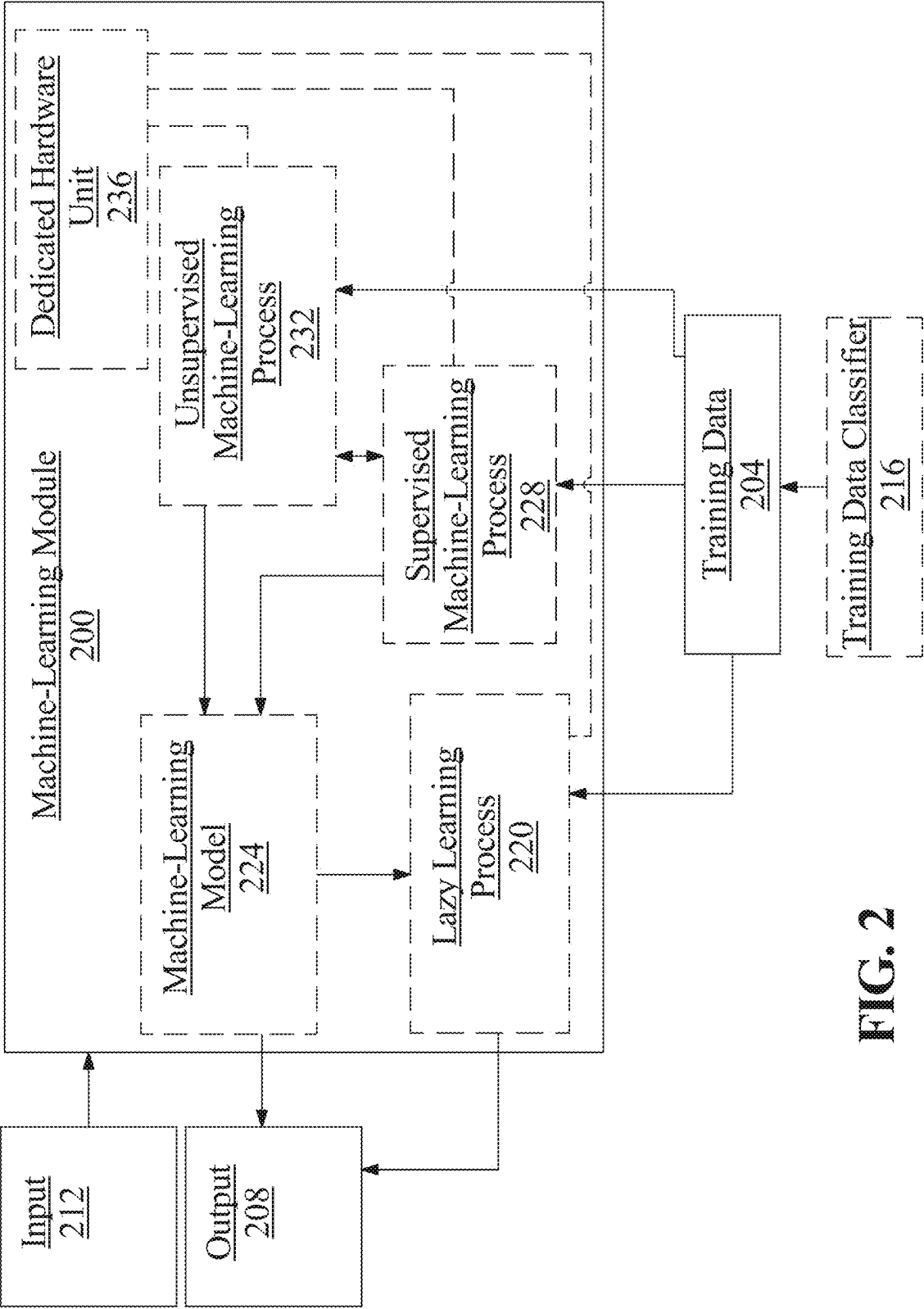


FIG. 2

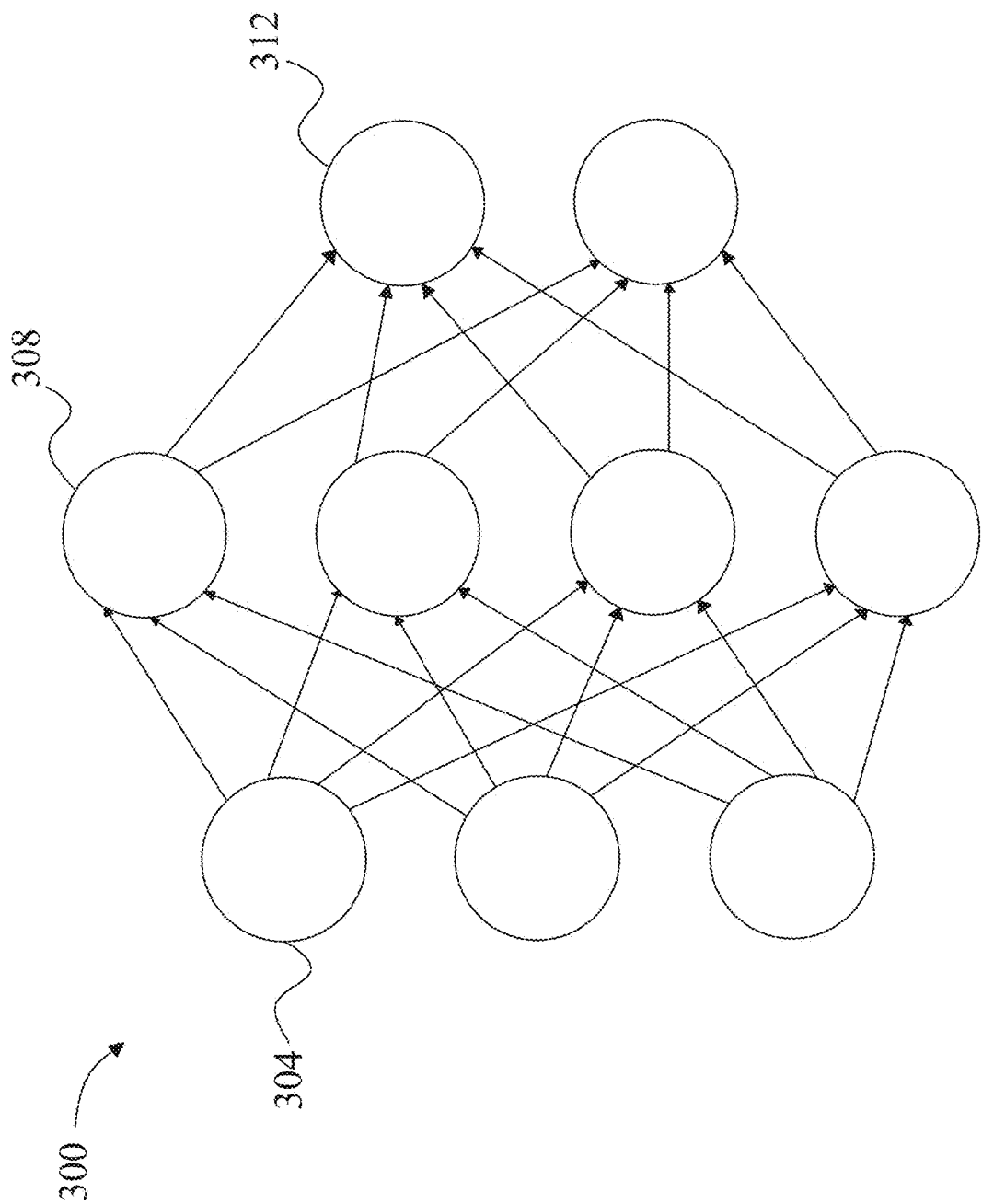


FIG. 3

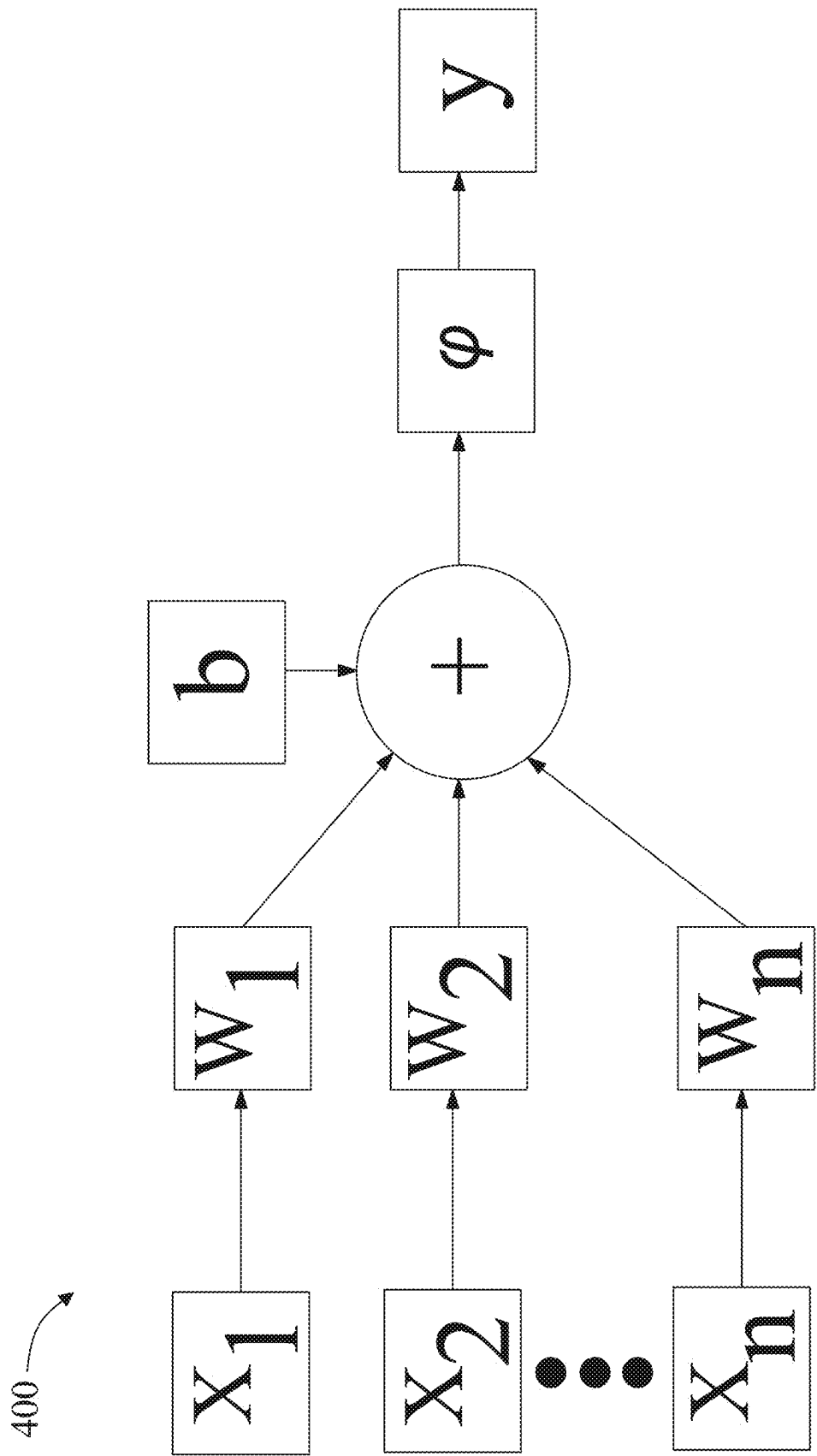


FIG. 4

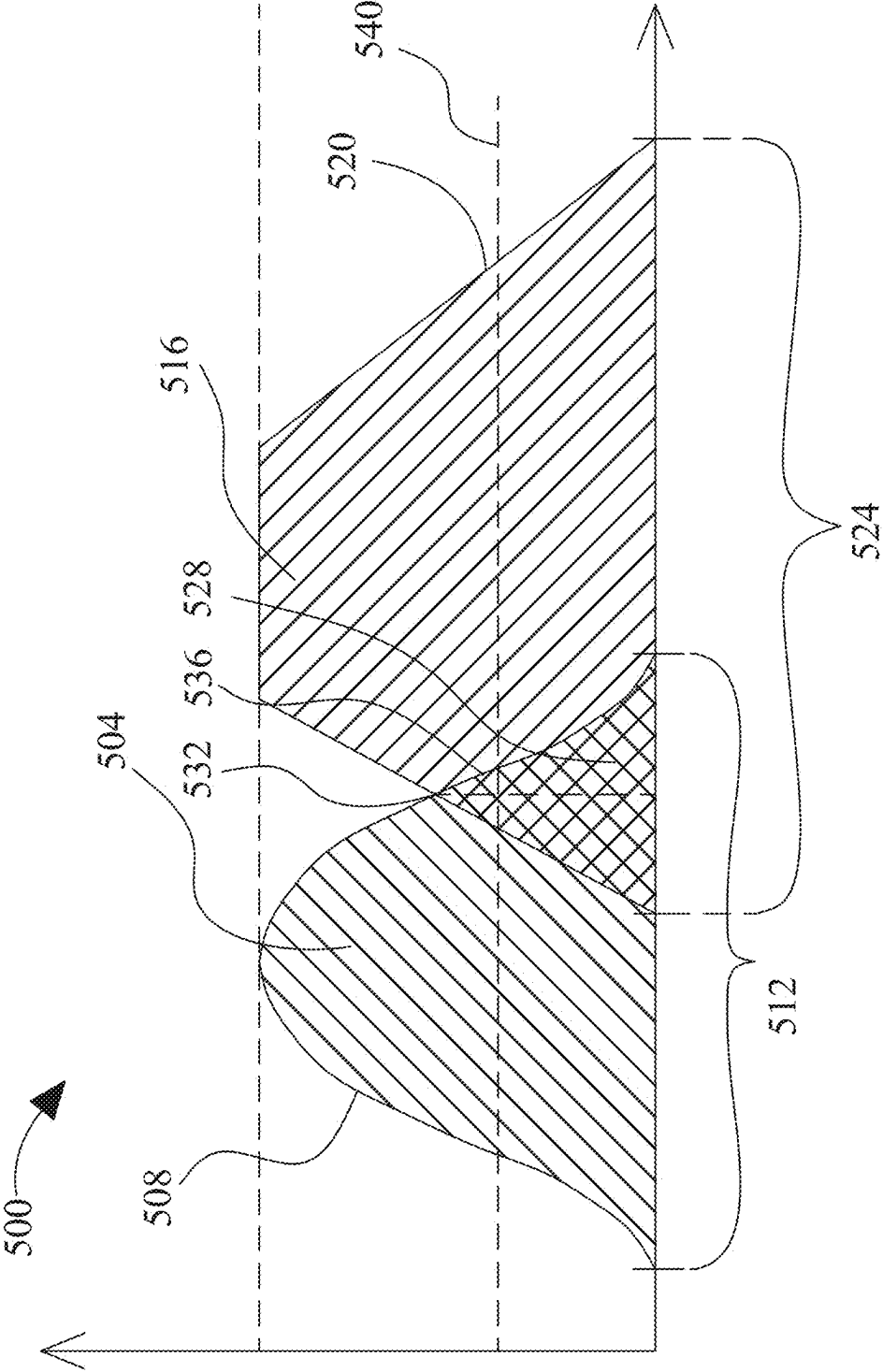
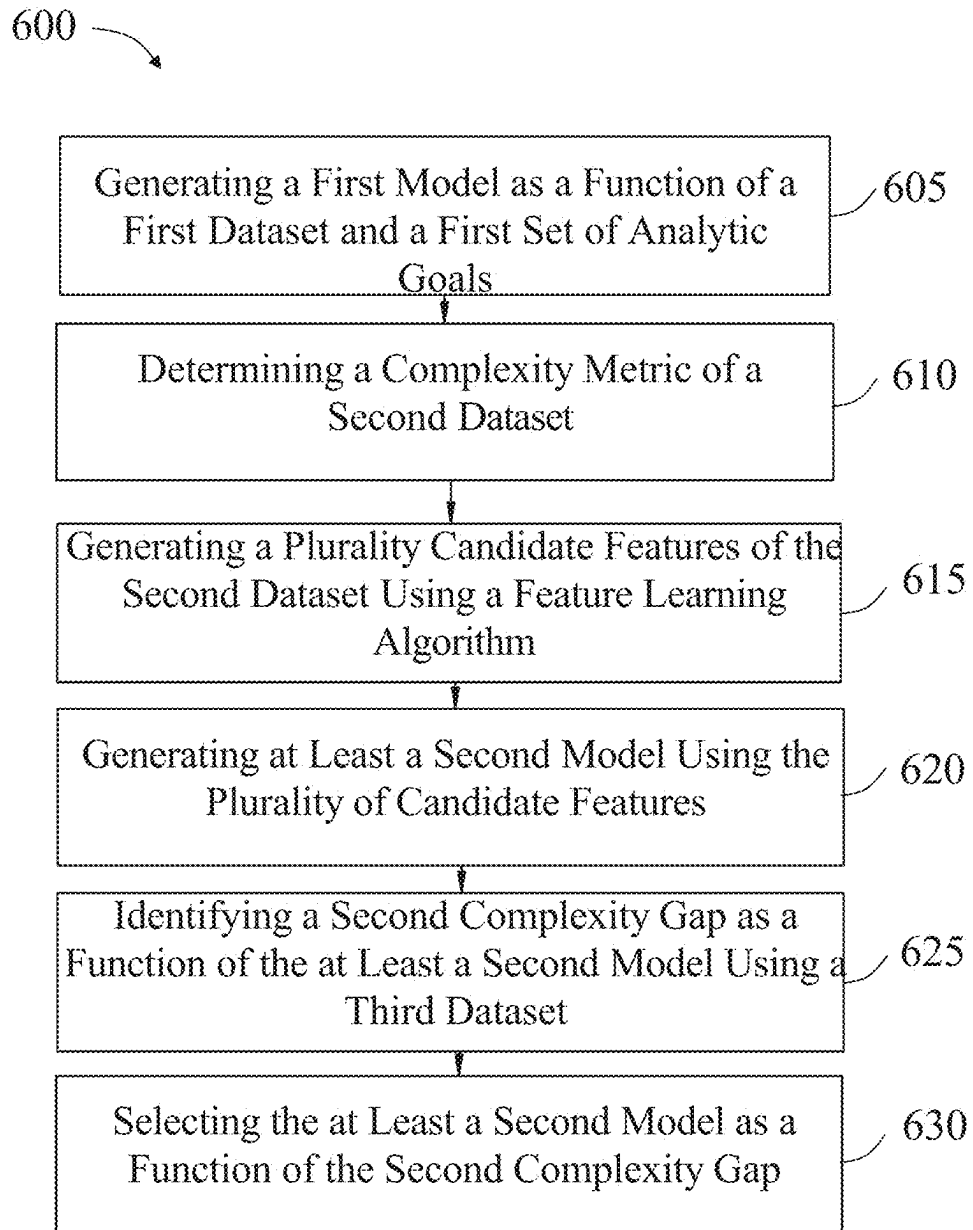


FIG. 5

**FIG. 6**

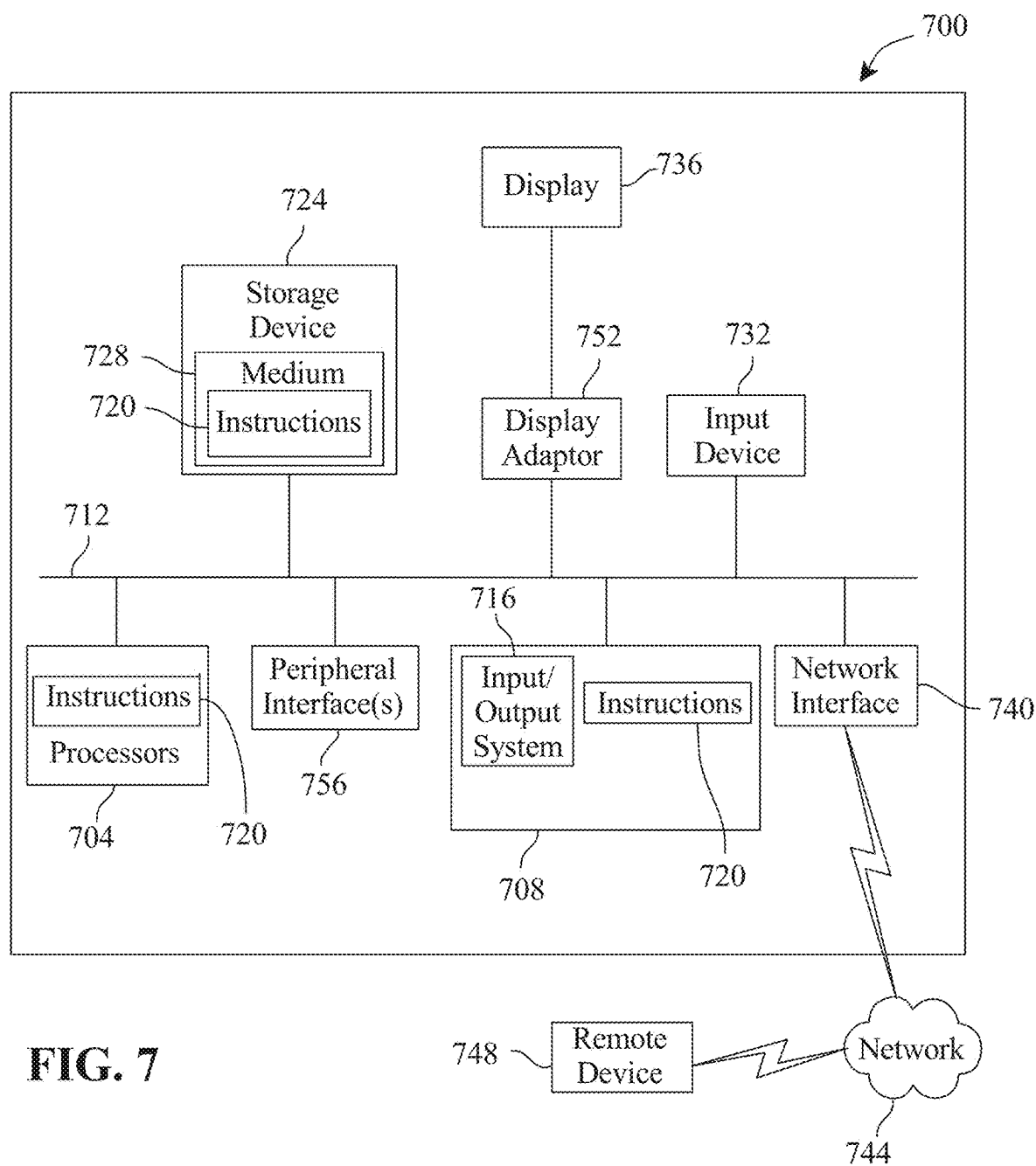


FIG. 7

**SYSTEM AND METHODS FOR AN
ADAPTIVE MACHINE LEARNING MODEL
SELECTION BASED ON DATA
COMPLEXITY AND USER GOALS**

**CROSS-REFERENCE TO RELATED
APPLICATIONS**

[0001] This application is a continuation-in-part of Non-provisional application Ser. No. 18/600,487 filed on Mar. 8, 2024, and entitled “SYSTEM AND METHODS FOR AN ADAPTIVE MACHINE LEARNING MODEL SELECTION BASED ON DATA COMPLEXITY AND USER GOALS,” the entirety of which is incorporated herein by reference.

FIELD OF THE INVENTION

[0002] The present invention generally relates to the field of machine learning. In particular, the present invention is directed to a system and methods for an adaptive machine learning model selection based on data complexity and user goals.

BACKGROUND

[0003] Processes to be analyzed using machine learning and/or other artificial intelligence models can change over time. In particular, there are types of processes that become more complex as time passes. In such cases, a model trained at an earlier stage may be unable to converge, even if retrained.

SUMMARY OF THE DISCLOSURE

[0004] In an aspect, an apparatus for an adaptive machine learning model selection based on data complexity and user goals is described. The apparatus includes at least a computing device comprising at least a processor and a memory, the computing device configured to generate a first model as a function of a first dataset and a first set of analytic goals, the computing device configured to determine a complexity metric of a second dataset, the computing device configured to generate a plurality candidate features of the second dataset using a feature learning algorithm, the computing device configured to generate at least a second model using the plurality of candidate features, the computing device configured to identify a second complexity gap as a function of the at least a second model using a third dataset; and select the at least a second model as a function of the second complexity gap.

[0005] In another aspect, a method for an adaptive machine learning model selection based on data complexity and user goals is described. The method includes generating, using at least a processor, a first model as a function of a first dataset and a first set of analytic goals, determining, using at least a processor, a complexity metric of a second dataset, generating, using at least a processor, a plurality candidate features of the second dataset using a feature learning algorithm, generating, using at least a processor, at least a second model using the plurality of candidate features, identifying, using at least a processor, a second complexity gap as a function of the at least a second model using a third dataset, and selecting, using at least a processor, the at least a second model as a function of the second complexity gap.

[0006] These and other aspects and features of non-limiting embodiments of the present invention will become

apparent to those skilled in the art upon review of the following description of specific non-limiting embodiments of the invention in conjunction with the accompanying drawings.

BRIEF DESCRIPTION OF THE DRAWINGS

[0007] For the purpose of illustrating the invention, the drawings show aspects of one or more embodiments of the invention. However, it should be understood that the present invention is not limited to the precise arrangements and instrumentalities shown in the drawings, wherein:

[0008] FIG. 1 is a block diagram of an exemplary apparatus for an adaptive machine learning model selection based on data complexity and user goals;

[0009] FIG. 2 is a block diagram of an exemplary machine-learning process;

[0010] FIG. 3 is a diagram of an exemplary embodiment of neural network;

[0011] FIG. 4 is a diagram of an exemplary embodiment of a node of a neural network;

[0012] FIG. 5 is a diagram of an exemplary embodiment of a fuzzy set;

[0013] FIG. 6 is a flow diagram illustrating an exemplary work flow in one embodiment of the present invention; and

[0014] FIG. 7 is a block diagram of a computing system that can be used to implement any one or more of the methodologies disclosed herein and any one or more portions thereof.

[0015] The drawings are not necessarily to scale and may be illustrated by phantom lines, diagrammatic representations and fragmentary views. In certain instances, details that are not necessary for an understanding of the embodiments or that render other details difficult to perceive may have been omitted.

DETAILED DESCRIPTION

[0016] At a high level, aspects of the present disclosure are directed to systems and methods for an adaptive machine learning model selection based on data complexity and user goals. In an embodiment, this invention introduces an approach that integrates a user's goals with the underlying data's complexity. This harmonization empowers users to achieve more accurate and relevant outcomes, optimizing the machine learning model's efficiency. Among its many benefits, the apparatus ensures that models remain adaptable, alleviating stagnation due to over-reliance on a single model, thus fostering continuous innovation and growth.

[0017] Referring now to FIG. 1, an exemplary embodiment of an apparatus 100 for adaptive machine learning model selection based on data complexity and user goals is illustrated. Apparatus includes a computing device. Computing device includes a processor communicatively connected to a memory. As used in this disclosure, “communicatively connected” means connected by way of a connection, attachment or linkage between two or more relata which allows for reception and/or transmittance of information therebetween. For example, and without limitation, this connection may be wired or wireless, direct or indirect, and between two or more components, circuits, devices, systems, and the like, which allows for reception and/or transmittance of data and/or signal(s) therebetween. Data and/or signals therebetween may include, without limitation, electrical, electromagnetic, magnetic, video,

audio, radio and microwave data and/or signals, combinations thereof, and the like, among others. A communicative connection may be achieved, for example and without limitation, through wired or wireless electronic, digital or analog, communication, either directly or by way of one or more intervening devices or components. Further, communicative connection may include electrically coupling or connecting at least an output of one device, component, or circuit to at least an input of another device, component, or circuit. For example, and without limitation, via a bus or other facility for intercommunication between elements of a computing device. Communicative connecting may also include indirect connections via, for example and without limitation, wireless connection, radio communication, low power wide area network, optical communication, magnetic, capacitive, or optical coupling, and the like. In some instances, the terminology “communicatively coupled” may be used in place of communicatively connected in this disclosure.

[0018] Further referring to FIG. 1, Computing device may include any computing device as described in this disclosure, including without limitation a microcontroller, micro-processor, digital signal processor (DSP) and/or system on a chip (SoC) as described in this disclosure. Computing device may include, be included in, and/or communicate with a mobile device such as a mobile telephone or smartphone. Computing device may include a single computing device operating independently, or may include two or more computing device operating in concert, in parallel, sequentially or the like; two or more computing devices may be included together in a single computing device or in two or more computing devices. Computing device may interface or communicate with one or more additional devices as described below in further detail via a network interface device. Network interface device may be utilized for connecting computing device to one or more of a variety of networks, and one or more devices. Examples of a network interface device include, but are not limited to, a network interface card (e.g., a mobile network interface card, a LAN card), a modem, and any combination thereof. Examples of a network include, but are not limited to, a wide area network (e.g., the Internet, an enterprise network), a local area network (e.g., a network associated with an office, a building, a campus or other relatively small geographic space), a telephone network, a data network associated with a telephone/voice provider (e.g., a mobile communications provider data and/or voice network), a direct connection between two computing devices, and any combinations thereof. A network may employ a wired and/or a wireless mode of communication. In general, any network topology may be used. Information (e.g., data, software etc.) may be communicated to and/or from a computer and/or a computing device. Computing device may include but is not limited to, for example, a computing device or cluster of computing devices in a first location and a second computing device or cluster of computing devices in a second location. Computing device may include one or more computing devices dedicated to data storage, security, distribution of traffic for load balancing, and the like. Computing device may distribute one or more computing tasks as described below across a plurality of computing devices of computing device, which may operate in parallel, in series, redundantly, or in any other manner used for distribution of tasks or memory

between computing devices. Computing device may be implemented, as a non-limiting example, using a “shared nothing” architecture.

[0019] With continued reference to FIG. 1, computing device may be designed and/or configured to perform any method, method step, or sequence of method steps in any embodiment described in this disclosure, in any order and with any degree of repetition. For instance, computing device may be configured to perform a single step or sequence repeatedly until a desired or commanded outcome is achieved; repetition of a step or a sequence of steps may be performed iteratively and/or recursively using outputs of previous repetitions as inputs to subsequent repetitions, aggregating inputs and/or outputs of repetitions to produce an aggregate result, reduction or decrement of one or more variables such as global variables, and/or division of a larger processing task into a set of iteratively addressed smaller processing tasks. Computing device may perform any step or sequence of steps as described in this disclosure in parallel, such as simultaneously and/or substantially simultaneously performing a step two or more times using two or more parallel threads, processor cores, or the like; division of tasks between parallel threads and/or processes may be performed according to any protocol suitable for division of tasks between iterations. Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various ways in which steps, sequences of steps, processing tasks, and/or data may be subdivided, shared, or otherwise dealt with using iteration, recursion, and/or parallel processing.

[0020] With continued reference to FIG. 1, an apparatus **100** includes a memory **108** communicatively connected to at least a processor **104**. For the purposes of this disclosure, “communicatively connected” means connected by way of a connection, attachment or linkage between two or more relata which allows for reception and/or transmittance of information therebetween. For example, and without limitation, this connection may be wired or wireless, direct or indirect, and between two or more components, circuits, devices, systems, and the like, which allows for reception and/or transmittance of data and/or signal(s) therebetween. Data and/or signals therebetween may include, without limitation, electrical, electromagnetic, magnetic, video, audio, radio and microwave data and/or signals, combinations thereof, and the like, among others. A communicative connection may be achieved, for example and without limitation, through wired or wireless electronic, digital or analog, communication, either directly or by way of one or more intervening devices or components. Further, communicative connection may include electrically coupling or connecting at least an output of one device, component, or circuit to at least an input of another device, component, or circuit. For example, and without limitation, via a bus or other facility for intercommunication between elements of a computing device. Communicative connecting may also include indirect connections via, for example and without limitation, wireless connection, radio communication, low power wide area network, optical communication, magnetic, capacitive, or optical coupling, and the like. In some instances, the terminology “communicatively coupled” may be used in place of communicatively connected in this disclosure.

[0021] With continued reference to FIG. 1, apparatus **100** and/or computing device may perform determinations, clas-

sification, and/or analysis steps, methods, processes, or the like as described in this disclosure using machine learning processes. A “machine learning process,” as used in this disclosure, is a process that automatically uses a body of data known as “training data” and/or a “training set” (described further below) to generate an algorithm that will be performed by a computing device/module to produce outputs given data provided as inputs; this is in contrast to a non-machine learning software program where the commands to be executed are determined in advance by a user and written in a programming language. Machine-learning process may utilize supervised, unsupervised, lazy-learning processes and/or neural networks, described further below.

[0022] A “classifier,” as used in this disclosure is a machine-learning model, such as a mathematical model, neural net, or program generated by a machine learning algorithm known as a “classification algorithm,” as described in further detail below, that sorts inputs into categories or bins of data, outputting the categories or bins of data and/or labels associated therewith. A classifier may be configured to output at least a datum that labels or otherwise identifies a set of data that are clustered together, found to be close under a distance metric as described below, or the like. Computing device **104** and/or another device may generate a classifier using a classification algorithm, defined as a processes whereby a computing device **104** derives a classifier from training data. Classification may be performed using, without limitation, linear classifiers such as without limitation logistic regression and/or naive Bayes classifiers, nearest neighbor classifiers such as k-nearest neighbors classifiers, support vector machines, least squares support vector machines, fisher’s linear discriminant, quadratic classifiers, decision trees, boosted trees, random forest classifiers, learning vector quantization, and/or neural network-based classifiers.

[0023] Still referring to FIG. 1, computing device **104** may be configured to generate a classifier using a Naïve Bayes classification algorithm. Naïve Bayes classification algorithm generates classifiers by assigning class labels to problem instances, represented as vectors of element values. Class labels are drawn from a finite set. Naïve Bayes classification algorithm may include generating a family of algorithms that assume that the value of a particular element is independent of the value of any other element, given a class variable. Naïve Bayes classification algorithm may be based on Bayes Theorem expressed as $P(A/B)=P(B/A)P(A)=P(B)$, where $P(A/B)$ is the probability of hypothesis A given data B also known as posterior probability; $P(B/A)$ is the probability of data B given that the hypothesis A was true; $P(A)$ is the probability of hypothesis A being true regardless of data also known as prior probability of A; and $P(B)$ is the probability of the data regardless of the hypothesis. A naïve Bayes algorithm may be generated by first transforming training data into a frequency table. Computing device **104** may then calculate a likelihood table by calculating probabilities of different data entries and classification labels. Computing device **104** may utilize a naïve Bayes equation to calculate a posterior probability for each class. A class containing the highest posterior probability is the outcome of prediction. Naïve Bayes classification algorithm may include a gaussian model that follows a normal distribution. Naïve Bayes classification algorithm may include a multinomial model that is used for discrete counts.

Naïve Bayes classification algorithm may include a Bernoulli model that may be utilized when vectors are binary.

[0024] With continued reference to FIG. 1, computing device **104** may be configured to generate a classifier using a K-nearest neighbors (KNN) algorithm. A “K-nearest neighbors algorithm” as used in this disclosure, includes a classification method that utilizes feature similarity to analyze how closely out-of-sample-features resemble training data to classify input data to one or more clusters and/or categories of features as represented in training data; this may be performed by representing both training data and input data in vector forms, and using one or more measures of vector similarity to identify classifications within training data, and to determine a classification of input data. K-nearest neighbors algorithm may include specifying a K-value, or a number directing the classifier to select the k most similar entries training data to a given sample, determining the most common classifier of the entries in the database, and classifying the known sample; this may be performed recursively and/or iteratively to generate a classifier that may be used to classify input data as further samples. For instance, an initial set of samples may be performed to cover an initial heuristic and/or “first guess” at an output and/or relationship, which may be seeded, without limitation, using expert input received according to any process as described herein. As a non-limiting example, an initial heuristic may include a ranking of associations between inputs and elements of training data. Heuristic may include selecting some number of highest-ranking associations and/or training data elements.

[0025] With continued reference to FIG. 1, generating k-nearest neighbors algorithm may generate a first vector output containing a data entry cluster, generating a second vector output containing an input data, and calculate the distance between the first vector output and the second vector output using any suitable norm such as cosine similarity, Euclidean distance measurement, or the like. Each vector output may be represented, without limitation, as an n-tuple of values, where n is at least two values. Each value of n-tuple of values may represent a measurement or other quantitative value associated with a given category of data, or attribute, examples of which are provided in further detail below; a vector may be represented, without limitation, in n-dimensional space using an axis per category of value represented in n-tuple of values, such that a vector has a geometric direction characterizing the relative quantities of attributes in the n-tuple as compared to each other. Two vectors may be considered equivalent where their directions, and/or the relative quantities of values within each vector as compared to each other, are the same; thus, as a non-limiting example, a vector represented as [5, 10, 15] may be treated as equivalent, for purposes of this disclosure, as a vector represented as [1, 2, 3]. Vectors may be more similar where their directions are more similar, and more different where their directions are more divergent; however, vector similarity may alternatively or additionally be determined using averages of similarities between like attributes, or any other measure of similarity suitable for any n-tuple of values, or aggregation of numerical similarity measures for the purposes of loss functions as described in further detail below. Any vectors as described herein may be scaled, such that each vector represents each attribute along an equivalent scale of values. Each vector may be “normalized,” or divided by a “length” attribute, such as a length attribute/as

derived using a Pythagorean norm: $l = \sum_{i=0}^n a_i^2$, where a_i is attribute number i of the vector. Scaling and/or normalization may function to make vector comparison independent of absolute quantities of attributes, while preserving any dependency on similarity of attributes; this may, for instance, be advantageous where cases represented in training data are represented by different quantities of samples, which may result in proportionally equivalent vectors with divergent values.

[0026] With continued reference to FIG. 1, apparatus **100** for an adaptive machine learning model selection based on data complexity and user goals, wherein the apparatus comprises at least one processor **108** and a memory **112** communicatively connected to the at least a processor **108**, wherein the memory **112** containing instructions configuring the at least processor **108** to generate a first model **116** as a function of a first dataset **120** and a first set of analytic goals **124**. As used in this disclosure, a “first model” is an initial computational framework or algorithm trained using data and specific techniques to derive insights, make predictions, or categorize data. The first model may serve as the baseline or reference for subsequent model iterations or comparisons. First model may be a machine-learning model, and may be trained using a first dataset **120**. Examples of the first model include but are not limited to a linear regression model trained to predict sales based on historical data, a decision tree model used to classify customer segments based on purchasing behavior, and a neural network initialized with random weights, ready to be trained on image data for object recognition. Processor **108** may use algorithms to create the first model **116**. The choice of algorithm may depend on the analytic goals set forth. For example, if the goal is to predict a continuous variable (like sales revenue), a regression algorithm might be used. If the goal is to classify data into categories, a classification algorithm would be more appropriate. The first dataset **120** is then fed into this algorithm to train first model, adjusting its parameters to best fit the data. First model may include a neural network, machine-learning model, or the like.

[0027] With continued reference to FIG. 1, computing device **104** may configure to incorporate the first dataset **120**. As used in this disclosure, a “first dataset” is the initial collection of data points, variables, or observations used to train, test, or validate the first model. The first dataset may provide foundational knowledge or context to the machine learning apparatus. Examples of first dataset includes but not limit to historical sales data from a company’s past fiscal years, patient medical records to predict disease outcomes, and a collection of images labeled for object recognition tasks. Depending on the goals, different types of data pre-processing might be applied. For instance, if the aim is to predict sales, historical sales data along with other relevant factors (like marketing spending, seasonality, etc.) would be included in the dataset. The goals help in refining the model. They can determine the evaluation metrics (like accuracy, precision, recall) and provide a benchmark to measure the model’s success. In the context of the Ceiling of Complexity, it is essential to be adaptable. As goals are achieved, new ones should be set to ensure continuous growth and prevent stagnation. In a non-limiting example, first dataset **120** may include the 4Cs (Commitment, Courage, Capability, and Confidence) and the 4 Freedoms (Freedom of Time, Freedom of Money, Freedom of Relationship, and Freedom of Purpose). Without limitation, each of the 4Cs may be

quantified through metrics such as the number of goals set and achieved (Commitment), instances of risk-taking or new initiatives started (Courage), training hours or certifications obtained (Capability), and self-assessed confidence levels before and after specific achievements (Confidence). Continuing, these metrics may provide a comprehensive view of how individuals or organizations are progressing through personal or professional development stages and overcoming the ceiling of complexity.

[0028] Still referring to FIG. 1, the at least a processor **108** configured to determine a complexity metric **132** of a second dataset **128**. As used in this disclosure, a complexity “metric” is a number, a vector or other data structure that could be compared to other numbers, vectors, and/or data structures, for vectors that may be compared geometrically or element by element, a mapping to a set membership such as a fuzzy set or a label from a classifier, or the like. Memory **112** may store the second dataset **128**, dataset includes but not limited to user uploads, sensors, APIs, or other data sources. Processor may compute statistical metrics like variance, entropy, or other measures that can provide insight into the data’s distribution and variability. As a non-limiting example, if complexity metric is a vector, processor may transform data features into vector space (like in the case of word embeddings or vector space models) and assess the geometrical properties of these vectors. For example, techniques like cosine similarity, Euclidean distance, or other distance measures can be used to compare vectors. For fuzzy set analysis, processor may employ fuzzy logic algorithms to map data points to certain fuzzy sets. The degree to which data points belong to a particular set or category can provide a measure of its complexity. In another example, if the metric is based on labels from a classifier, the processor might utilize a machine learning classifier to categorize the data. The classifier’s scores or error rates can then act as indicators of the dataset’s complexity. Once determined, complexity metric **132** may be stored in memory **112**. Complexity metric **132** can be retrieved for future comparisons or for making decisions related to model selection, training, or other operations. Processor **108** may use the complexity metric **132** to make adaptive decisions, possibly altering the approach to model selection or data preprocessing based on the assessed complexity of second dataset **128**.

[0029] With continued reference to FIG. 1, in an embodiment, wherein determining complexity metric includes numbers, vectors, and mapping to a set membership. Consider second dataset **128** is a collection of sales data for an e-commerce company over the past year. As used in this disclosure, “second dataset” may be an updated first dataset. A simple numerical complexity metric might be the variance in monthly sales. High variance indicates fluctuating sales, possibly implying complexity in consumer behavior, seasonality, or promotional effectiveness. In a non-limiting example, a dataset of daily temperatures in a city over a year, the range (difference between highest and lowest temperatures) could be used as a numeric complexity metric. A larger range indicates greater weather variability. In another example, the realm of Natural Language Processing (NLP), consider the second dataset **128** as a collection of customer reviews. Each review is converted into a vector using techniques like TF-IDF or Word2Vec. To gauge complexity, the average cosine similarity between vectors can be assessed. Low average cosine similarity indicates diverse sentiments and topics, hence higher complexity. In another

example, suppose the second dataset **128** contains data about people's goal-reaching habits. Using a fuzzy clustering algorithm, each person's goal preferences can be mapped to genres like short term, long term, or future as memberships in fuzzy sets. Someone might belong 0.7 to short-term, 0.5 to long-term, and 0.2 to future. The overall complexity metric can be the average fuzziness across all members, indicating how varied and overlapping the goal preferences are across the population. In a non-limiting example, second dataset **128** may include the 4Cs (Commitment, Courage, Capability, and Confidence) and the 4 Freedoms (Freedom of Time, Freedom of Money, Freedom of Relationship, and Freedom of Purpose). For example, without limitation the complexity metric may be the variance in how individuals score across these dimensions, with higher variance indicating greater diversity in development stages and resource availability among the individuals. For instance, without limitation, Commitment may be assessed by the number of projects completed on time, Courage by the number of innovative ideas proposed, Capability by the number of new skills acquired, and Confidence by self-reported levels of job satisfaction. Similarly, without limitation, Freedom of Time may be measured by the amount of time saved through delegation, Freedom of Money by investments in productivity tools, Freedom of Relationship by the quality of professional networks, and Freedom of Purpose by the alignment of daily activities with long-term goals.

[0030] Still referring to FIG. 1, the at least a processor **108** configured to compare the complexity metric with a predetermined threshold **136**, wherein the level of complexity comprises a scale of threshold **140**. As used in this disclosure, a "predetermined threshold" is a reference value or set of values against which the complexity metric is evaluated; a predetermined threshold may include any variable type and/or data structure suitable for use as a complexity metric. It may be a single number, a set or vector of numbers, allowing for nuanced comparisons such as measuring the vector proximity from the metric to this vector. The threshold could also be represented as a fuzzy set, facilitating comparisons that consider degrees of membership rather than binary in/out determinations. Additionally, predetermined threshold **136** may be a matching classifier label or a "centroid", indicative of techniques like k-means clustering where data is grouped around central points. Furthermore, the comparison of complexity metric **132** to predetermined threshold **136** does not have to be a straightforward mathematical comparison; it could be performed by a machine learning model itself, where both the metric and threshold are inputted, and the model outputs a comparison result. In an embodiment, wherein predetermined threshold is configured to select from a group. As used in this disclosure, a "group" refers to a collection or set of similar or related items, values, entities, or elements that are assembled or categorized based on certain criteria or characteristics. The group can represent a range, a cluster, or a category of predefined values or entities which the system can refer to or select from, depending on the context or application. In a non-limiting example, a professional may aim to increase their productivity at work. They may identify key performance indicators (KPIs) that reflect their productivity, such as the number of tasks completed, hours worked, and positive feedback received. Each of these KPIs can have a predetermined threshold grouped into categories like "beginner", "intermediate", and "expert". For instance,

completing 5 tasks a day might be categorized under "beginner", 10 tasks under "intermediate", and 20 tasks under "expert". As the professional progresses and completes more tasks daily, they move through these categories in the group. Predetermined threshold **136** may be selected from the group, may help the individual recognize their current performance level and gives them a clear goal to strive for, motivating them to reach the next category.

[0031] In another embodiment, wherein the group comprises a number, a set of numbers, a vector of numbers, a fuzzy set, a matching classifier label, and a centroid derived from K-means clustering. In a non-limiting example, a company may aim to optimize its marketing strategies to reach a broader customer base and increase sales. To measure the efficacy of their strategies, they may deploy a machine learning model that evaluates various metrics related to customer engagement, such as website visits, click-through rates, and conversion rates.

[0032] In an additional embodiment, wherein the group comprises a number, a set of numbers, a vector of numbers, a fuzzy set, a matching classifier label, and a centroid derived from K-means clustering. In a non-limiting example, a fitness tech startup may develop a wearable device to help users achieve their health and fitness goals. The wearable collects data like heart rate, steps taken, sleep quality, and more. The startup employs a machine learning model to provide actionable insights to users.

[0033] With continued reference to FIG. 1, the at least a processor **108** configured to identify a first complexity gap **144** as a function of the comparison and the first model **116**. As used in this disclosure, a "first complexity gap" is the difference or divergence between the complexity metric derived from the data and the capability or sophistication of the first model **116**. The first complexity gap **144** may be visualized as the space or distance between the complexity inherent in the dataset and the model's ability to understand, capture, or predict based on that complexity. It may signify how well or poorly the model is suited to the current data's intricacy, potentially influencing the model's performance and predictive accuracy. In a non-limiting example, a user with a smart fitness tracker may aim to become a long-distance runner. The first model **116** might include running a marathon in under 4 hours. User's current record for a marathon is 5 hours, the target set in the fitness tracker is to complete a marathon in under 4 hours. Complexity metric in the tracker may measure various metrics like average pace, heart rate, stride length, and stamina level, and determines that user's current average pace is 7 minutes per kilometer. The tracker, based on data from professional runners, may determine that to achieve a sub-4-hour marathon, the average pace needs to be approximately 5 minutes 40 seconds per kilometer. This would be the difference between user's current pace (7 minutes) and the target pace (5 minutes 40 seconds). In this case, the first complexity gap is 1 minute 20 seconds per kilometer. Identifying this complexity gap allows user to tailor training specifically to improve pace. It may provide a clear direction on where efforts need to be concentrated, allowing for more efficient and directed training.

[0034] With continued reference to FIG. 1, the at least a processor **108** configured to generate a plurality candidate features **148** of the second dataset using a feature learning algorithm **152**. As used in this disclosure, a "candidate feature" is a potential attribute or characteristic derived from

the original dataset that might be able to predict or relevance to a given task or objective. These features may be generated through algorithms or methods that sift through the dataset to extract, construct, or transform data in ways that may enhance the model's understanding or representation of the underlying problem. In the context of goal reaching, candidate features may be a stepping stones or indicators that can assist in achieving the primary objective. In a non-limiting example, a user who may be aspired to become a team leader within the next year. To understand the attributes of successful team leaders, a software tool analyzes various profiles of team leaders within the company and across similar industries. Second dataset may consist of data profiles of various employees, including their education, years of experience, skills, projects handled, feedback scores, and other related attributes. The software uses a feature learning algorithm to identify and extract potentially relevant characteristics that correlate with being a successful team leader. The algorithm might find that individuals with a management degree or certification are more likely to become team leaders. In an embodiment, wherein generating the feature learning algorithm further includes an unsupervised machine learning algorithm, a supervised machine learning algorithm, a semi-supervised machine learning algorithm and a reinforcement learning algorithm. Unsupervised machine learning algorithm works with datasets that do not have labeled responses. It may try to identify patterns or groupings within the data on its own. For example, a user aiming to decide a major. The user uses an app that applies unsupervised learning on academic performance, extracurricular activities, and personal interests. The algorithm clusters similar subjects and activities together. Through this, it may identify a strong inclination towards humanities and arts in the profile, assisting in narrowing down choices for a major. Supervised machine learning algorithm requires a dataset where the "answer" or outcome to a problem is already known. It learns from this data to make predictions or inferences about new, unseen data. User may use a tool that employs supervised learning, which has data of accepted student profiles from previous years. By inputting user's grades, extracurriculars, and other achievements, the tool predicts the chances of getting an acceptance, guiding the user on areas the user might need to improve. Semi-supervised machine learning algorithm uses both labeled and unlabeled data for training. It may be beneficial when acquiring a fully labeled dataset is expensive or time-consuming. For example, a user may want to be a novelist and is using a software to gauge the appeal of her writings. The software has a few labeled datasets (categorized as 'good' or 'bad') and many unlabeled writings. Using semi-supervised learning, it identifies patterns from labeled data and applies them to unlabeled writings, giving the user feedback on the appeal of the writing pieces. In reinforcement learning, an agent learns how to behave in an environment by taking certain actions and receiving rewards or penalties in return. A user dreams of becoming a chess grandmaster. A chess simulation game that may apply reinforcement learning. With every move the user makes, apparatus provides feedback, rewarding good moves and penalizing bad ones. Over time, the user may learn optimal strategies, adapting and refining gameplay, inching closer to the goal of mastering chess. Each of these algorithms offers a different approach to understanding data and making decisions. As used in this disclosure, they may provide

insights, feedback, and directions based on the data at hand, assisting individuals in making informed choices and strategies towards their objectives.

[0035] With continued reference to FIG. 1, the at least a processor **108** configured to generate at least a second model using the plurality of candidate features. As used in this disclosure, a "second model" is a subsequent or refined iteration of a machine learning or computational model that has been generated based on updated, additional, or refined features, often to enhance accuracy, precision, or to cater to a different set of requirements than the initial model. For example, the user's first model might be based on generic training schedules for marathon runners. This model suggests the user run a certain number of miles each day, follow a specified diet, and get a set amount of rest. After a few weeks, the user uses a fitness tracker to monitor various metrics like heart rate, pace, calorie burn, and recovery times. From this data, new insights and patterns emerge, indicating which parts of the training are effective and which areas need modification. Based on this new data (plurality of candidate features), second model of training is developed. Second model might suggest the user incorporate interval training, adjust the diet to include more protein, and increase rest days. Second model is tailored specifically to the user's needs, with the goal of improving marathon performance. The development of second model exemplifies the iterative nature of goal-reaching. Just as the user refines the training strategy based on new data and insights, machine learning models can be retrained and refined based on new features to improve outcomes. In an embodiment, wherein second model comprises a second set of candidate features distinct from a first set of candidates with the first model. This may incorporate a different set of features or attributes when compared to first model. These distinct features can lead to a different understanding, pattern recognition, or prediction capability for the model, helping it cater to evolving needs or insights.

[0036] With continued reference to FIG. 1, the at least a processor **108** configured to identify a second complexity gap **156** as a function of the at least a second model **160** using a third dataset **164**, and select the at least a second model as a function of the second complexity gap. To provide a comprehensive evaluation, processor **108** may use a third dataset **164** to measure the efficacy of at least a second model **160**. The choice of the most suitable second model is heavily influenced by the value of the second complexity gap. Essentially, this gap may offer insights into how well each model performs in comparison to the desired or expected outcomes, and the model with the most optimal performance is selected. Furthermore, process of determining or measuring second complexity gap **156** may be executed through machine learning methodologies. For instance, machine learning can aid in analyzing patterns, predicting outcomes, or even enhancing the precision of results. On the other side, non-ML alternatives could involve statistical analysis, heuristic methods, or rule-based evaluations. In addition to selecting an optimal model based on the complexity gap, there's flexibility in the approach. For example, instead of choosing between different models, apparatus might decide between different sets of candidate features. This way, it may emphasize the significance of the attributes that the model uses rather than the model itself. In an embodiment, wherein comparing between complexity metric **132** and predetermined threshold **136** may be con-

figured to receive complexity metric **132** and predetermined threshold **136** as input and output a comparison result. For example, a user striving to enhance their productivity levels. Complexity metric may quantify the user's daily tasks, interruptions faced, time spent on each task, and the overall quality of completed work. Predetermined threshold may be a predefined standard of productivity that the user aspires to reach, encompassing metrics like a specific number of tasks to complete, limited interruptions, efficient time management, and a high-quality standard for finished tasks. Upon feeding these values (complexity metric and predetermined threshold) into the comparison module of a machine learning model may predict the user's end-of-day productivity based on historical data and current inputs. It then contrasts this prediction with the predetermined threshold to determine if the user is on track. The output, or the comparison result, will provide a clear indication of where the user stands in relation to their productivity goal and can serve as a guide for any required course corrections. In another embodiment, wherein the at least a processor **108** is further configured to generate multiple second models **160**, wherein the multiple second models **160** comprises a distinct set of candidate feature **148**. In an additional embodiment, wherein the at least a processor **108** is further configured to derive a performance score. This score is essentially a quantifiable metric that may signify the effectiveness, accuracy, or utility of a particular function or operation being executed within apparatus **100**. Performance score may be derived from various parameters such as accuracy, speed, reliability, error rate, among others. The derivation of the performance score can be achieved through multiple methods. For instance, using machine learning models, apparatus can predict the performance based on historical data and current parameters. Alternatively, through non-ML approaches, mathematical formulas or algorithm-based calculations can be applied based on predefined standards or benchmarks. In a non-limiting example, second model **160**, utilizing the third dataset **164**, may be particularly effective in determining the stage of growth of goals by analyzing patterns in the data. For instance, without limitation, utilizing machine learning algorithms, second model **160** may identify milestones and progress markers within the dataset. Continuing, this may allow processor **108** to measure the current state of goal achievement and predict future growth stages. For example, without limitation, in the context of sales data for an e-commerce company, second model **160** may track metrics such as sales volume growth, customer acquisition rates, and seasonal performance variations to identify whether the company is in an early growth phase, a rapid expansion phase, or a maturity phase. Additionally, processor **108** may include non-ML methods like statistical analysis and heuristic evaluations to further refine the understanding of growth stages. In a non-limiting example, by comparing complexity metric **132** with predetermined thresholds **136**, the system may output a comparison result that highlights areas of performance that meet or exceed expectations versus those that need improvement. Continuing, this approach may enable second model **160** to be adaptable and provide insights at various stages of goal progression. For example, without limitation, if predetermined threshold **136** is based on achieving a specific sales target, the comparison result may indicate how close the company is to reaching this target and suggest adjustments in strategies to accelerate growth.

[0037] With continued reference to FIG. 1, at least a processor **108** may dynamically update predetermined threshold **136**. Without limitation, at least a processor **108** may continuously or periodically collect and analyze data as described herein, such as first dataset **120**, second dataset **128**, and third dataset **164**. In a non-limiting example, at least a processor **108** may analyze the data in real-time or at set intervals and may identify patterns and trends using statistical methods, machine learning algorithms, and the like. Continuing, the analysis may evaluate whether the current threshold remains optimal by comparing the current operational conditions against predefined criteria or benchmarks. Continuing, if the performance deviates from desired levels, it may indicate that the threshold needs adjustment. Continuing, when at least a processor **108** determines that an update is necessary, at least a processor **108** may calculate a new threshold value, considering the latest data trends and ensuring that the new threshold aligns with the optimal operational parameters of apparatus **100**. In a non-limiting example, at least a processor **108** may update predetermined threshold **136** immediately or during a scheduled maintenance window.

[0038] Referring now to FIG. 2, an exemplary embodiment of a machine-learning module **200** that may perform one or more machine-learning processes as described in this disclosure is illustrated. Machine-learning module may perform determinations, classification, and/or analysis steps, methods, processes, or the like as described in this disclosure using machine learning processes. A "machine learning process," as used in this disclosure, is a process that automatically uses training data **204** to generate an algorithm instantiated in hardware or software logic, data structures, and/or functions that will be performed by a computing device/module to produce outputs **208** given data provided as inputs **212**; this is in contrast to a non-machine learning software program where the commands to be executed are determined in advance by a user and written in a programming language.

[0039] Still referring to FIG. 2, "training data," as used herein, is data containing correlations that a machine-learning process may use to model relationships between two or more categories of data elements. For instance, and without limitation, training data **204** may include a plurality of data entries, also known as "training examples," each entry representing a set of data elements that were recorded, received, and/or generated together; data elements may be correlated by shared existence in a given data entry, by proximity in a given data entry, or the like. Multiple data entries in training data **204** may evince one or more trends in correlations between categories of data elements; for instance, and without limitation, a higher value of a first data element belonging to a first category of data element may tend to correlate to a higher value of a second data element belonging to a second category of data element, indicating a possible proportional or other mathematical relationship linking values belonging to the two categories. Multiple categories of data elements may be related in training data **204** according to various correlations; correlations may indicate causative and/or predictive links between categories of data elements, which may be modeled as relationships such as mathematical relationships by machine-learning processes as described in further detail below. Training data **204** may be formatted and/or organized by categories of data elements, for instance by associating data elements with one

or more descriptors corresponding to categories of data elements. As a non-limiting example, training data **204** may include data entered in standardized forms by persons or processes, such that entry of a given data element in a given field in a form may be mapped to one or more descriptors of categories. Elements in training data **204** may be linked to descriptors of categories by tags, tokens, or other data elements; for instance, and without limitation, training data **204** may be provided in fixed-length formats, formats linking positions of data to categories such as comma-separated value (CSV) formats and/or self-describing formats such as extensible markup language (XML), JavaScript Object Notation (JSON), or the like, enabling processes or devices to detect categories of data.

[0040] Alternatively or additionally, and continuing to refer to FIG. 2, training data **204** may include one or more elements that are not categorized; that is, training data **204** may not be formatted or contain descriptors for some elements of data. Machine-learning algorithms and/or other processes may sort training data **204** according to one or more categorizations using, for instance, natural language processing algorithms, tokenization, detection of correlated values in raw data and the like; categories may be generated using correlation and/or other processing algorithms. As a non-limiting example, in a corpus of text, phrases making up a number “n” of compound words, such as nouns modified by other nouns, may be identified according to a statistically significant prevalence of n-grams containing such words in a particular order; such an n-gram may be categorized as an element of language such as a “word” to be tracked similarly to single words, generating a new category as a result of statistical analysis. Similarly, in a data entry including some textual data, a person’s name may be identified by reference to a list, dictionary, or other compendium of terms, permitting ad-hoc categorization by machine-learning algorithms, and/or automated association of data in the data entry with descriptors or into a given format. The ability to categorize data entries automatically may enable the same training data **204** to be made applicable for two or more distinct machine-learning algorithms as described in further detail below. Training data **204** used by machine-learning module **200** may correlate any input data as described in this disclosure to any output data as described in this disclosure. As a non-limiting illustrative example, the input data may include various sales metrics such as the number of customers visiting a store, the time spent by each customer in the store, the products viewed, and past purchasing behavior. The output data, in turn, could be predictions about future purchasing behavior, suggesting which products a customer is likely to buy next, or identifying potential high-value customers based on their interactions. This correlation enables businesses to tailor their marketing efforts and inventory decisions based on predicted customer behavior, thereby optimizing sales and improving overall customer satisfaction. The machine-learning module **200** can continuously learn and adapt from this training data, refining its predictions over time as more data is fed into the system.

[0041] Further referring to FIG. 2, training data may be filtered, sorted, and/or selected using one or more supervised and/or unsupervised machine-learning processes and/or models as described in further detail below; such models may include without limitation a training data classifier **216**. Training data classifier **216** may include a “classifier,” which as used in this disclosure is a machine-learning model as

defined below, such as a data structure representing and/or using a mathematical model, neural net, or program generated by a machine learning algorithm known as a “classification algorithm,” as described in further detail below, that sorts inputs into categories or bins of data, outputting the categories or bins of data and/or labels associated therewith. A classifier may be configured to output at least a datum that labels or otherwise identifies a set of data that are clustered together, found to be close under a distance metric as described below, or the like. A distance metric may include any norm, such as, without limitation, a Pythagorean norm. Machine-learning module **200** may generate a classifier using a classification algorithm, defined as a processes whereby a computing device and/or any module and/or component operating thereon derives a classifier from training data **204**. Classification may be performed using, without limitation, linear classifiers such as without limitation logistic regression and/or naive Bayes classifiers, nearest neighbor classifiers such as k-nearest neighbors classifiers, support vector machines, least squares support vector machines, fisher’s linear discriminant, quadratic classifiers, decision trees, boosted trees, random forest classifiers, learning vector quantization, and/or neural network-based classifiers. As a non-limiting example, training data classifier **216** may classify elements of training data to specific demographic characteristics, such as age group, geographic location, or past purchasing behavior. For instance, if a retailer is keen on understanding the buying habits of millennials in the Pacific Northwest region who have previously purchased eco-friendly products, the classifier can filter out training data specific to this cohort.

[0042] Still referring to FIG. 2, computing device **204** may be configured to generate a classifier using a Naïve Bayes classification algorithm. Naïve Bayes classification algorithm generates classifiers by assigning class labels to problem instances, represented as vectors of element values. Class labels are drawn from a finite set. Naïve Bayes classification algorithm may include generating a family of algorithms that assume that the value of a particular element is independent of the value of any other element, given a class variable. Naïve Bayes classification algorithm may be based on Bayes Theorem expressed as $P(A/B)=P(B/A)P(A)=P(B)$, where $P(A/B)$ is the probability of hypothesis A given data B also known as posterior probability; $P(B/A)$ is the probability of data B given that the hypothesis A was true; $P(A)$ is the probability of hypothesis A being true regardless of data also known as prior probability of A; and $P(B)$ is the probability of the data regardless of the hypothesis. A naïve Bayes algorithm may be generated by first transforming training data into a frequency table. Computing device **204** may then calculate a likelihood table by calculating probabilities of different data entries and classification labels. Computing device **204** may utilize a naïve Bayes equation to calculate a posterior probability for each class. A class containing the highest posterior probability is the outcome of prediction. Naïve Bayes classification algorithm may include a gaussian model that follows a normal distribution. Naïve Bayes classification algorithm may include a multinomial model that is used for discrete counts. Naïve Bayes classification algorithm may include a Bernoulli model that may be utilized when vectors are binary.

[0043] With continued reference to FIG. 2, computing device **204** may be configured to generate a classifier using a K-nearest neighbors (KNN) algorithm. A “K-nearest

neighbors algorithm” as used in this disclosure, includes a classification method that utilizes feature similarity to analyze how closely out-of-sample-features resemble training data to classify input data to one or more clusters and/or categories of features as represented in training data; this may be performed by representing both training data and input data in vector forms, and using one or more measures of vector similarity to identify classifications within training data, and to determine a classification of input data. K-nearest neighbors algorithm may include specifying a K-value, or a number directing the classifier to select the k most similar entries training data to a given sample, determining the most common classifier of the entries in the database, and classifying the known sample; this may be performed recursively and/or iteratively to generate a classifier that may be used to classify input data as further samples. For instance, an initial set of samples may be performed to cover an initial heuristic and/or “first guess” at an output and/or relationship, which may be seeded, without limitation, using expert input received according to any process as described herein. As a non-limiting example, an initial heuristic may include a ranking of associations between inputs and elements of training data. Heuristic may include selecting some number of highest-ranking associations and/or training data elements.

[0044] With continued reference to FIG. 2, generating k-nearest neighbors algorithm may generate a first vector output containing a data entry cluster, generating a second vector output containing an input data, and calculate the distance between the first vector output and the second vector output using any suitable norm such as cosine similarity, Euclidean distance measurement, or the like. Each vector output may be represented, without limitation, as an n-tuple of values, where n is at least two values. Each value of n-tuple of values may represent a measurement or other quantitative value associated with a given category of data, or attribute, examples of which are provided in further detail below; a vector may be represented, without limitation, in n-dimensional space using an axis per category of value represented in n-tuple of values, such that a vector has a geometric direction characterizing the relative quantities of attributes in the n-tuple as compared to each other. Two vectors may be considered equivalent where their directions, and/or the relative quantities of values within each vector as compared to each other, are the same; thus, as a non-limiting example, a vector represented as [5, 10, 15] may be treated as equivalent, for purposes of this disclosure, as a vector represented as [1, 2, 3]. Vectors may be more similar where their directions are more similar, and more different where their directions are more divergent; however, vector similarity may alternatively or additionally be determined using averages of similarities between like attributes, or any other measure of similarity suitable for any n-tuple of values, or aggregation of numerical similarity measures for the purposes of loss functions as described in further detail below. Any vectors as described herein may be scaled, such that each vector represents each attribute along an equivalent scale of values. Each vector may be “normalized,” or divided by a “length” attribute, such as a length attribute/as derived using a Pythagorean norm: $l = \sum_{i=0}^n a_i^2$, where a_i is attribute number i of the vector. Scaling and/or normalization may function to make vector comparison independent of absolute quantities of attributes, while preserving any dependency on similarity of attributes; this may, for

instance, be advantageous where cases represented in training data are represented by different quantities of samples, which may result in proportionally equivalent vectors with divergent values.

[0045] With further reference to FIG. 2, training examples for use as training data may be selected from a population of potential examples according to cohorts relevant to an analytical problem to be solved, a classification task, or the like. Alternatively or additionally, training data may be selected to span a set of likely circumstances or inputs for a machine-learning model and/or process to encounter when deployed. For instance, and without limitation, for each category of input data to a machine-learning process or model that may exist in a range of values in a population of phenomena such as images, user data, process data, physical data, or the like, a computing device, processor, and/or machine-learning model may select training examples representing each possible value on such a range and/or a representative sample of values on such a range. Selection of a representative sample may include selection of training examples in proportions matching a statistically determined and/or predicted distribution of such values according to relative frequency, such that, for instance, values encountered more frequently in a population of data so analyzed are represented by more training examples than values that are encountered less frequently. Alternatively or additionally, a set of training examples may be compared to a collection of representative values in a database and/or presented to a user, so that a process can detect, automatically or via user input, one or more values that are not included in the set of training examples. Computing device, processor, and/or module may automatically generate a missing training example; this may be done by receiving and/or retrieving a missing input and/or output value and correlating the missing input and/or output value with a corresponding output and/or input value collocated in a data record with the retrieved value, provided by a user and/or other device, or the like.

[0046] Continuing to refer to FIG. 2, computer, processor, and/or module may be configured to preprocess training data. “Preprocessing” training data, as used in this disclosure, is transforming training data from raw form to a format that can be used for training a machine learning model. Preprocessing may include sanitizing, feature selection, feature scaling, data augmentation and the like.

[0047] Still referring to FIG. 2, computer, processor, and/or module may be configured to sanitize training data. “Sanitizing” training data, as used in this disclosure, is a process whereby training examples are removed that interfere with convergence of a machine-learning model and/or process to a useful result. For instance, and without limitation, a training example may include an input and/or output value that is an outlier from typically encountered values, such that a machine-learning algorithm using the training example will be adapted to an unlikely amount as an input and/or output; a value that is more than a threshold number of standard deviations away from an average, mean, or expected value, for instance, may be eliminated. Alternatively or additionally, one or more training examples may be identified as having poor quality data, where “poor quality” is defined as having a signal to noise ratio below a threshold value. Sanitizing may include steps such as removing duplicative or otherwise redundant data, interpolating missing data, correcting data errors, standardizing data, identifying

outliers, and the like. In a nonlimiting example, sanitization may include utilizing algorithms for identifying duplicate entries or spell-check algorithms.

[0048] As a non-limiting example, and with further reference to FIG. 2, images used to train an image classifier or other machine-learning model and/or process that takes images as inputs or generates images as outputs may be rejected if image quality is below a threshold value. For instance, and without limitation, computing device, processor, and/or module may perform blur detection, and eliminate one or more Blur detection may be performed, as a non-limiting example, by taking Fourier transform, or an approximation such as a Fast Fourier Transform (FFT) of the image and analyzing a distribution of low and high frequencies in the resulting frequency-domain depiction of the image; numbers of high-frequency values below a threshold level may indicate blurriness. As a further non-limiting example, detection of blurriness may be performed by convolving an image, a channel of an image, or the like with a Laplacian kernel; this may generate a numerical score reflecting a number of rapid changes in intensity shown in the image, such that a high score indicates clarity and a low score indicates blurriness. Blurriness detection may be performed using a gradient-based operator, which measures operators based on the gradient or first derivative of an image, based on the hypothesis that rapid changes indicate sharp edges in the image, and thus are indicative of a lower degree of blurriness. Blur detection may be performed using Wavelet-based operator, which takes advantage of the capability of coefficients of the discrete wavelet transform to describe the frequency and spatial content of images. Blur detection may be performed using statistics-based operators take advantage of several image statistics as texture descriptors in order to compute a focus level. Blur detection may be performed by using discrete cosine transform (DCT) coefficients in order to compute a focus level of an image from its frequency content.

[0049] Continuing to refer to FIG. 2, computing device, processor, and/or module may be configured to precondition one or more training examples. For instance, and without limitation, where a machine learning model and/or process has one or more inputs and/or outputs requiring, transmitting, or receiving a certain number of bits, samples, or other units of data, one or more training examples' elements to be used as or compared to inputs and/or outputs may be modified to have such a number of units of data. For instance, a computing device, processor, and/or module may convert a smaller number of units, such as in a low pixel count image, into a desired number of units, for instance by upsampling and interpolating. As a non-limiting example, a low pixel count image may have 100 pixels, however a desired number of pixels may be 128. Processor may interpolate the low pixel count image to convert the 100 pixels into 128 pixels. It should also be noted that one of ordinary skill in the art, upon reading this disclosure, would know the various methods to interpolate a smaller number of data units such as samples, pixels, bits, or the like to a desired number of such units. In some instances, a set of interpolation rules may be trained by sets of highly detailed inputs and/or outputs and corresponding inputs and/or outputs downsampled to smaller numbers of units, and a neural network or other machine learning model that is trained to predict interpolated pixel values using the training data. As a non-limiting example, a sample input and/or output, such

as a sample picture, with sample-expanded data units (e.g., pixels added between the original pixels) may be input to a neural network or machine-learning model and output a pseudo replica sample-picture with dummy values assigned to pixels between the original pixels based on a set of interpolation rules. As a non-limiting example, in the context of an image classifier, a machine-learning model may have a set of interpolation rules trained by sets of highly detailed images and images that have been downsampled to smaller numbers of pixels, and a neural network or other machine learning model that is trained using those examples to predict interpolated pixel values in a facial picture context. As a result, an input with sample-expanded data units (the ones added between the original data units, with dummy values) may be run through a trained neural network and/or model, which may fill in values to replace the dummy values. Alternatively or additionally, processor, computing device, and/or module may utilize sample expander methods, a low-pass filter, or both. As used in this disclosure, a "low-pass filter" is a filter that passes signals with a frequency lower than a selected cutoff frequency and attenuates signals with frequencies higher than the cutoff frequency. The exact frequency response of the filter depends on the filter design. Computing device, processor, and/or module may use averaging, such as luma or chroma averaging in images, to fill in data units in between original data units.

[0050] In some embodiments, and with continued reference to FIG. 2, computing device, processor, and/or module may down-sample elements of a training example to a desired lower number of data elements. As a non-limiting example, a high pixel count image may have 256 pixels, however a desired number of pixels may be 128. Processor may down-sample the high pixel count image to convert the 256 pixels into 128 pixels. In some embodiments, processor may be configured to perform downsampling on data. Downsampling, also known as decimation, may include removing every Nth entry in a sequence of samples, all but every Nth entry, or the like, which is a process known as "compression," and may be performed, for instance by an N-sample compressor implemented using hardware or software. Anti-aliasing and/or anti-imaging filters, and/or low-pass filters, may be used to clean up side-effects of compression.

[0051] Further referring to FIG. 2, feature selection includes narrowing and/or filtering training data to exclude features and/or elements, or training data including such elements, that are not relevant to a purpose for which a trained machine-learning model and/or algorithm is being trained, and/or collection of features and/or elements, or training data including such elements, on the basis of relevance or utility for an intended task or purpose for a trained machine-learning model and/or algorithm is being trained. Feature selection may be implemented, without limitation, using any process described in this disclosure, including without limitation using training data classifiers, exclusion of outliers, or the like.

[0052] With continued reference to FIG. 2, feature scaling may include, without limitation, normalization of data entries, which may be accomplished by dividing numerical fields by norms thereof, for instance as performed for vector normalization. Feature scaling may include absolute maximum scaling, wherein each quantitative datum is divided by the maximum absolute value of all quantitative data of a set or subset of quantitative data. Feature scaling may include

min-max scaling, in which each value X has a minimum value X_{min} in a set or subset of values subtracted therefrom, with the result divided by the range of the values, give maximum value in the set or subset X_{max} :

$$X_{new} = \frac{X - X_{min}}{X_{max} - X_{min}}.$$

Feature scaling may include mean normalization, which involves use of a mean value of a set and/or subset of values, X_{mean} with maximum and minimum values:

$$X_{new} = \frac{X - X_{mean}}{X_{max} - X_{min}}.$$

Feature scaling may include standardization, where a difference between X and X_{mean} is divided by a standard deviation σ of a set or subset of values:

$$X_{new} = \frac{X - X_{mean}}{\sigma}.$$

Scaling may be performed using a median value of a set or subset X_{median} and/or interquartile range (IQR), which represents the difference between the 25th percentile value and the 50th percentile value (or closest values thereto by a rounding protocol), such as:

$$X_{new} = \frac{X - X_{median}}{IQR}.$$

Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various alternative or additional approaches that may be used for feature scaling.

[0053] Further referring to FIG. 2, computing device, processor, and/or module may be configured to perform one or more processes of data augmentation. “Data augmentation” as used in this disclosure is addition of data to a training set using elements and/or entries already in the dataset. Data augmentation may be accomplished, without limitation, using interpolation, generation of modified copies of existing entries and/or examples, and/or one or more generative AI processes, for instance using deep neural networks and/or generative adversarial networks; generative processes may be referred to alternatively in this context as “data synthesis” and as creating “synthetic data.” Augmentation may include performing one or more transformations on data, such as geometric, color space, affine, brightness, cropping, and/or contrast transformations of images.

[0054] Still referring to FIG. 2, machine-learning module **200** may be configured to perform a lazy-learning process **220** and/or protocol, which may alternatively be referred to as a “lazy loading” or “call-when-needed” process and/or protocol, may be a process whereby machine learning is conducted upon receipt of an input to be converted to an output, by combining the input and training set to derive the algorithm to be used to produce the output on demand. For instance, an initial set of simulations may be performed to cover an initial heuristic and/or “first guess” at an output

and/or relationship. As a non-limiting example, an initial heuristic may include a ranking of associations between inputs and elements of training data **204**. Heuristic may include selecting some number of highest-ranking associations and/or training data **204** elements. Lazy learning may implement any suitable lazy learning algorithm, including without limitation a K-nearest neighbors algorithm, a lazy naïve Bayes algorithm, or the like; persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various lazy-learning algorithms that may be applied to generate outputs as described in this disclosure, including without limitation lazy learning applications of machine-learning algorithms as described in further detail below.

[0055] Alternatively or additionally, and with continued reference to FIG. 2, machine-learning processes as described in this disclosure may be used to generate machine-learning models **224**. A “machine-learning model,” as used in this disclosure, is a data structure representing and/or instantiating a mathematical and/or algorithmic representation of a relationship between inputs and outputs, as generated using any machine-learning process including without limitation any process as described above, and stored in memory; an input is submitted to a machine-learning model **224** once created, which generates an output based on the relationship that was derived. For instance, and without limitation, a linear regression model, generated using a linear regression algorithm, may compute a linear combination of input data using coefficients derived during machine-learning processes to calculate an output datum. As a further non-limiting example, a machine-learning model **224** may be generated by creating an artificial neural network, such as a convolutional neural network comprising an input layer of nodes, one or more intermediate layers, and an output layer of nodes. Connections between nodes may be created via the process of “training” the network, in which elements from a training data **204** set are applied to the input nodes, a suitable training algorithm (such as Levenberg-Marquardt, conjugate gradient, simulated annealing, or other algorithms) is then used to adjust the connections and weights between nodes in adjacent layers of the neural network to produce the desired values at the output nodes. This process is sometimes referred to as deep learning.

[0056] Still referring to FIG. 2, machine-learning algorithms may include at least a supervised machine-learning process **228**. At least a supervised machine-learning process **228**, as defined herein, include algorithms that receive a training set relating a number of inputs to a number of outputs, and seek to generate one or more data structures representing and/or instantiating one or more mathematical relations relating inputs to outputs, where each of the one or more mathematical relations is optimal according to some criterion specified to the algorithm using some scoring function. For instance, a supervised learning algorithm may include inputs as described in this disclosure as inputs, outputs as described in this disclosure as outputs, and a scoring function representing a desired form of relationship to be detected between inputs and outputs; scoring function may, for instance, seek to maximize the probability that a given input and/or combination of elements inputs is associated with a given output to minimize the probability that a given input is not associated with a given output. Scoring function may be expressed as a risk function representing an “expected loss” of an algorithm relating inputs to outputs, where loss is computed as an error function representing a

degree to which a prediction generated by the relation is incorrect when compared to a given input-output pair provided in training data **204**. Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various possible variations of at least a supervised machine-learning process **228** that may be used to determine relation between inputs and outputs. Supervised machine-learning processes may include classification algorithms as defined above.

[0057] With further reference to FIG. 2, training a supervised machine-learning process may include, without limitation, iteratively updating coefficients, biases, weights based on an error function, expected loss, and/or risk function. For instance, an output generated by a supervised machine-learning model using an input example in a training example may be compared to an output example from the training example; an error function may be generated based on the comparison, which may include any error function suitable for use with any machine-learning algorithm described in this disclosure, including a square of a difference between one or more sets of compared values or the like. Such an error function may be used in turn to update one or more weights, biases, coefficients, or other parameters of a machine-learning model through any suitable process including without limitation gradient descent processes, least-squares processes, and/or other processes described in this disclosure. This may be done iteratively and/or recursively to gradually tune such weights, biases, coefficients, or other parameters. Updating may be performed, in neural networks, using one or more back-propagation algorithms. Iterative and/or recursive updates to weights, biases, coefficients, or other parameters as described above may be performed until currently available training data is exhausted and/or until a convergence test is passed, where a “convergence test” is a test for a condition selected as indicating that a model and/or weights, biases, coefficients, or other parameters thereof has reached a degree of accuracy. A convergence test may, for instance, compare a difference between two or more successive errors or error function values, where differences below a threshold amount may be taken to indicate convergence. Alternatively or additionally, one or more errors and/or error function values evaluated in training iterations may be compared to a threshold.

[0058] Still referring to FIG. 2, a computing device, processor, and/or module may be configured to perform method, method step, sequence of method steps and/or algorithm described in reference to this figure, in any order and with any degree of repetition. For instance, a computing device, processor, and/or module may be configured to perform a single step, sequence and/or algorithm repeatedly until a desired or commanded outcome is achieved; repetition of a step or a sequence of steps may be performed iteratively and/or recursively using outputs of previous repetitions as inputs to subsequent repetitions, aggregating inputs and/or outputs of repetitions to produce an aggregate result, reduction or decrement of one or more variables such as global variables, and/or division of a larger processing task into a set of iteratively addressed smaller processing tasks. A computing device, processor, and/or module may perform any step, sequence of steps, or algorithm in parallel, such as simultaneously and/or substantially simultaneously performing a step two or more times using two or more parallel threads, processor cores, or the like; division of

tasks between parallel threads and/or processes may be performed according to any protocol suitable for division of tasks between iterations. Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various ways in which steps, sequences of steps, processing tasks, and/or data may be subdivided, shared, or otherwise dealt with using iteration, recursion, and/or parallel processing.

[0059] Further referring to FIG. 2, machine learning processes may include at least an unsupervised machine-learning processes **232**. An unsupervised machine-learning process, as used herein, is a process that derives inferences in datasets without regard to labels; as a result, an unsupervised machine-learning process may be free to discover any structure, relationship, and/or correlation provided in the data. Unsupervised processes **232** may not require a response variable; unsupervised processes **232** may be used to find interesting patterns and/or inferences between variables, to determine a degree of correlation between two or more variables, or the like.

[0060] Still referring to FIG. 2, machine-learning module **200** may be designed and configured to create a machine-learning model **224** using techniques for development of linear regression models. Linear regression models may include ordinary least squares regression, which aims to minimize the square of the difference between predicted outcomes and actual outcomes according to an appropriate norm for measuring such a difference (e.g., a vector-space distance norm); coefficients of the resulting linear equation may be modified to improve minimization. Linear regression models may include ridge regression methods, where the function to be minimized includes the least-squares function plus term multiplying the square of each coefficient by a scalar amount to penalize large coefficients. Linear regression models may include least absolute shrinkage and selection operator (LASSO) models, in which ridge regression is combined with multiplying the least-squares term by a factor of 1 divided by double the number of samples. Linear regression models may include a multi-task lasso model wherein the norm applied in the least-squares term of the lasso model is the Frobenius norm amounting to the square root of the sum of squares of all terms. Linear regression models may include the elastic net model, a multi-task elastic net model, a least angle regression model, a LARS lasso model, an orthogonal matching pursuit model, a Bayesian regression model, a logistic regression model, a stochastic gradient descent model, a perceptron model, a passive aggressive algorithm, a robustness regression model, a Huber regression model, or any other suitable model that may occur to persons skilled in the art upon reviewing the entirety of this disclosure. Linear regression models may be generalized in an embodiment to polynomial regression models, whereby a polynomial equation (e.g., a quadratic, cubic or higher-order equation) providing a best predicted output/actual output fit is sought; similar methods to those described above may be applied to minimize error functions, as will be apparent to persons skilled in the art upon reviewing the entirety of this disclosure.

[0061] Continuing to refer to FIG. 2, machine-learning algorithms may include, without limitation, linear discriminant analysis. Machine-learning algorithm may include quadratic discriminant analysis. Machine-learning algorithms may include kernel ridge regression. Machine-learning algorithms may include support vector machines, including

without limitation support vector classification-based regression processes. Machine-learning algorithms may include stochastic gradient descent algorithms, including classification and regression algorithms based on stochastic gradient descent. Machine-learning algorithms may include nearest neighbors algorithms. Machine-learning algorithms may include various forms of latent space regularization such as variational regularization. Machine-learning algorithms may include Gaussian processes such as Gaussian Process Regression. Machine-learning algorithms may include cross-decomposition algorithms, including partial least squares and/or canonical correlation analysis. Machine-learning algorithms may include naïve Bayes methods. Machine-learning algorithms may include algorithms based on decision trees, such as decision tree classification or regression algorithms. Machine-learning algorithms may include ensemble methods such as bagging meta-estimator, forest of randomized trees, AdaBoost, gradient tree boosting, and/or voting classifier methods. Machine-learning algorithms may include neural net algorithms, including convolutional neural net processes.

[0062] Still referring to FIG. 2, a machine-learning model and/or process may be deployed or instantiated by incorporation into a program, apparatus, system and/or module. For instance, and without limitation, a machine-learning model, neural network, and/or some or all parameters thereof may be stored and/or deployed in any memory or circuitry. Parameters such as coefficients, weights, and/or biases may be stored as circuit-based constants, such as arrays of wires and/or binary inputs and/or outputs set at logic “1” and “0” voltage levels in a logic circuit to represent a number according to any suitable encoding system including two's complement or the like or may be stored in any volatile and/or non-volatile memory. Similarly, mathematical operations and input and/or output of data to or from models, neural network layers, or the like may be instantiated in hardware circuitry and/or in the form of instructions in firmware, machine-code such as binary operation code instructions, assembly language, or any higher-order programming language. Any technology for hardware and/or software instantiation of memory, instructions, data structures, and/or algorithms may be used to instantiate a machine-learning process and/or model, including without limitation any combination of production and/or configuration of non-reconfigurable hardware elements, circuits, and/or modules such as without limitation ASICs, production and/or configuration of reconfigurable hardware elements, circuits, and/or modules such as without limitation FPGAs, production and/or of non-reconfigurable and/or configuration non-rewritable memory elements, circuits, and/or modules such as without limitation non-rewritable ROM, production and/or configuration of reconfigurable and/or rewritable memory elements, circuits, and/or modules such as without limitation rewritable ROM or other memory technology described in this disclosure, and/or production and/or configuration of any computing device and/or component thereof as described in this disclosure. Such deployed and/or instantiated machine-learning model and/or algorithm may receive inputs from any other process, module, and/or component described in this disclosure, and produce outputs to any other process, module, and/or component described in this disclosure.

[0063] Continuing to refer to FIG. 2, any process of training, retraining, deployment, and/or instantiation of any

machine-learning model and/or algorithm may be performed and/or repeated after an initial deployment and/or instantiation to correct, refine, and/or improve the machine-learning model and/or algorithm. Such retraining, deployment, and/or instantiation may be performed as a periodic or regular process, such as retraining, deployment, and/or instantiation at regular elapsed time periods, after some measure of volume such as a number of bytes or other measures of data processed, a number of uses or performances of processes described in this disclosure, or the like, and/or according to a software, firmware, or other update schedule. Alternatively or additionally, retraining, deployment, and/or instantiation may be event-based, and may be triggered, without limitation, by user inputs indicating sub-optimal or otherwise problematic performance and/or by automated field testing and/or auditing processes, which may compare outputs of machine-learning models and/or algorithms, and/or errors and/or error functions thereof, to any thresholds, convergence tests, or the like, and/or may compare outputs of processes described herein to similar thresholds, convergence tests or the like. Event-based retraining, deployment, and/or instantiation may alternatively or additionally be triggered by receipt and/or generation of one or more new training examples; a number of new training examples may be compared to a preconfigured threshold, where exceeding the preconfigured threshold may trigger retraining, deployment, and/or instantiation.

[0064] Still referring to FIG. 2, retraining and/or additional training may be performed using any process for training described above, using any currently or previously deployed version of a machine-learning model and/or algorithm as a starting point. Training data for retraining may be collected, preconditioned, sorted, classified, sanitized or otherwise processed according to any process described in this disclosure. Training data may include, without limitation, training examples including inputs and correlated outputs used, received, and/or generated from any version of any system, module, machine-learning model or algorithm, apparatus, and/or method described in this disclosure; such examples may be modified and/or labeled according to user feedback or other processes to indicate desired results, and/or may have actual or measured results from a process being modeled and/or predicted by system, module, machine-learning model or algorithm, apparatus, and/or method as “desired” results to be compared to outputs for training processes as described above.

[0065] Redeployment may be performed using any reconfiguring and/or rewriting of reconfigurable and/or rewritable circuit and/or memory elements; alternatively, redeployment may be performed by production of new hardware and/or software components, circuits, instructions, or the like, which may be added to and/or may replace existing hardware and/or software components, circuits, instructions, or the like.

[0066] Further referring to FIG. 2, one or more processes or algorithms described above may be performed by at least a dedicated hardware unit 236. A “dedicated hardware unit,” for the purposes of this figure, is a hardware component, circuit, or the like, aside from a principal control circuit and/or processor performing method steps as described in this disclosure, that is specifically designated or selected to perform one or more specific tasks and/or processes described in reference to this figure, such as without limitation preconditioning and/or sanitization of training data

and/or training a machine-learning algorithm and/or model. A dedicated hardware unit 236 may include, without limitation, a hardware unit that can perform iterative or massed calculations, such as matrix-based calculations to update or tune parameters, weights, coefficients, and/or biases of machine-learning models and/or neural networks, efficiently using pipelining, parallel processing, or the like; such a hardware unit may be optimized for such processes by, for instance, including dedicated circuitry for matrix and/or signal processing operations that includes, e.g., multiple arithmetic and/or logical circuit units such as multipliers and/or adders that can act simultaneously and/or in parallel or the like. Such dedicated hardware units 236 may include, without limitation, graphical processing units (GPUs), dedicated signal processing modules, FPGA or other reconfigurable hardware that has been configured to instantiate parallel processing units for one or more specific tasks, or the like. A computing device, processor, apparatus, or module may be configured to instruct one or more dedicated hardware units 236 to perform one or more operations described herein, such as evaluation of model and/or algorithm outputs, one-time or iterative updates to parameters, coefficients, weights, and/or biases, and/or any other operations such as vector and/or matrix operations as described in this disclosure.

[0067] Referring now to FIG. 3, an exemplary embodiment of neural network 300 is illustrated. A neural network 300 also known as an artificial neural network, is a network of “nodes,” or data structures having one or more inputs, one or more outputs, and a function determining outputs based on inputs. Such nodes may be organized in a network, such as without limitation a convolutional neural network, including an input layer of nodes 304, one or more intermediate layers 308, and an output layer of nodes 312. Connections between nodes may be created via the process of “training” the network, in which elements from a training dataset are applied to the input nodes, a suitable training algorithm (such as Levenberg-Marquardt, conjugate gradient, simulated annealing, or other algorithms) is then used to adjust the connections and weights between nodes in adjacent layers of the neural network to produce the desired values at the output nodes. This process is sometimes referred to as deep learning. Connections may run solely from input nodes toward output nodes in a “feed-forward” network, or may feed outputs of one layer back to inputs of the same or a different layer in a “recurrent network.” As a further non-limiting example, a neural network may include a convolutional neural network comprising an input layer of nodes, one or more intermediate layers, and an output layer of nodes. A “convolutional neural network,” as used in this disclosure, is a neural network in which at least one hidden layer is a convolutional layer that convolves inputs to that layer with a subset of inputs known as a “kernel,” along with one or more additional layers such as pooling layers, fully connected layers, and the like.

[0068] Referring now to FIG. 4, an exemplary embodiment of a node 400 of a neural network is illustrated. A node may include, without limitation, a plurality of inputs x_i that may receive numerical values from inputs to a neural network containing the node and/or from other nodes. Node may perform one or more activation functions to produce its output given one or more inputs, such as without limitation computing a binary step function comparing an input to a threshold value and outputting either a logic 1 or logic 0

output or something equivalent, a linear activation function whereby an output is directly proportional to the input, and/or a non-linear activation function, wherein the output is not proportional to the input. Non-linear activation functions may include, without limitation, a sigmoid function of the form

$$f(x) = \frac{1}{1 + e^{-x}}$$

given input x , a tan h (hyperbolic tangent) function, of the form

$$\frac{e^x - e^{-x}}{e^x + e^{-x}},$$

a tan h derivative function such as $f(x) = \tanh^2(x)$, a rectified linear unit function such as $f(x) = \max(0, x)$, a “leaky” and/or “parametric” rectified linear unit function such as $f(x) = \max(ax, x)$ for some a , an exponential linear units function such as

$$f(x) = \begin{cases} x & \text{for } x \geq 0 \\ \alpha(e^x - 1) & \text{for } x < 0 \end{cases}$$

for some value of α (this function may be replaced and/or weighted by its own derivative in some embodiments), a softmax function such as

$$f(x_i) = \frac{e^{x_i}}{\sum_j x_j}$$

where the inputs to an instant layer are x_i , a swish function such as $f(x) = x * \text{sigmoid}(x)$, a Gaussian error linear unit function such as $f(x) = a(1 + \tanh(\sqrt{2/\pi}(x + bx')))$ for some values of a , b , and r , and/or a scaled exponential linear unit function such as

$$f(x) = \lambda \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases} \quad (x)$$

Fundamentally, there is no limit to the nature of functions of inputs x_i that may be used as activation functions. As a non-limiting and illustrative example, node may perform a weighted sum of inputs using weights w_i that are multiplied by respective inputs x_i . Additionally or alternatively, a bias b may be added to the weighted sum of the inputs such that an offset is added to each unit in the neural network layer that is independent of the input to the layer. The weighted sum may then be input into a function ϕ , which may generate one or more outputs y . Weight w_i applied to an input x_i may indicate whether the input is “excitatory,” indicating that it has strong influence on the one or more outputs y , for instance by the corresponding weight having a large numerical value, and/or a “inhibitory,” indicating it has a weak effect influence on the one more inputs y , for instance by the

corresponding weight having a small numerical value. The values of weights w_i may be determined by training a neural network using training data, which may be performed using any suitable process as described above.

[0069] Referring to FIG. 5, an exemplary embodiment of fuzzy set comparison 500 is illustrated. A first fuzzy set 504 may be defined by a first membership function 508 that may assign a probability to each input value within a first range of values 512, indicating the degree which each input is considered a member of the first fuzzy set 504. The first membership function 508 may be scaled between 0 and 1, where the area under the curve of the function may represent the collective membership of values to first fuzzy set 504. The first range of values 512 may be depicted as a single-dimensional range for simplicity, first range of values 512 may extend into multi-dimensional space, encompassing various ranges, axes, or dimensions to capture more complex relationships or characteristics. First membership function 508 may include any suitable function mapping first range 512 to a probability interval, including without limitation a triangular function defined by two linear elements such as line segments or planes that intersect at or below the top of the probability interval. As a non-limiting example, triangular membership function may be defined as:

$$y(x, a, b, c) = \begin{cases} 0, & \text{for } x > c \text{ and } x < a \\ \frac{x-a}{b-a}, & \text{for } a \leq x < b \\ \frac{c-x}{c-b}, & \text{if } b < x \leq c \end{cases}$$

a trapezoidal membership function may be defined as:

$$y(x, a, b, c, d) = \max\left(\min\left(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}\right), 0\right)$$

a sigmoidal function may be defined as:

$$y(x, a, c) = \frac{1}{1 + e^{-a(x-c)}}$$

a Gaussian membership function may be defined as:

$$y(x, c, \sigma) = e^{-\frac{1}{2}\left(\frac{x-c}{\sigma}\right)^2}$$

and a bell membership function may be defined as:

$$y(x, a, b, c) = \left[1 + \left|\frac{x-c}{a}\right|^{2b}\right]^{-1}$$

Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various alternative or additional membership functions that may be used consistently with this disclosure.

[0070] Still referring to FIG. 5, first fuzzy set 504 may represent any value or combination of values as described above, including output from one or more machine-learning

models and biofeedback signals from sensor 108, a predetermined class, such as without limitation a user state (e.g., attentive, inattentive, and the like). A second fuzzy set 516, which may represent any value which may be represented by first fuzzy set 504, may be defined by a second membership function 520 on a second range 524; second range 524 may be identical and/or overlap with first range 512 and/or may be combined with first range via Cartesian product or the like to generate a mapping permitting evaluation overlap of first fuzzy set 504 and second fuzzy set 516. Where first fuzzy set 504 and second fuzzy set 516 have a region 528 that overlaps, first membership function 508 and second membership function 520 may intersect at a point 532 representing a probability, as defined on probability interval, of a match between first fuzzy set 504 and second fuzzy set 516. Alternatively or additionally, a single value of first and/or second fuzzy set may be located at a locus 536 on first range 512 and/or second range 524, where a probability of membership may be taken by evaluation of first membership function 508 and/or second membership function 520 at that range point. A probability at 528 and/or 532 may be compared to a threshold 540 to determine whether a positive match is indicated. Threshold 540 may, in a non-limiting example, represent a degree of match between first fuzzy set 504 and second fuzzy set 516, and/or single values therein with each other or with either set, which is sufficient for purposes of the matching process; for instance, threshold may indicate a sufficient degree of overlap between an output from one or more machine-learning models and/or a biofeedback signal and a predetermined class, such as without limitation a user state, for combination to occur as described above. Alternatively or additionally, each threshold may be tuned by a machine-learning and/or statistical process, for instance and without limitation as described in further detail below.

[0071] Further referring to FIG. 5, in an embodiment, a degree of match between fuzzy sets may be used to classify a biofeedback signal with a user state. For instance, if a biofeedback signal has a fuzzy set matching a user state fuzzy set by having a degree of overlap exceeding a threshold, computing device 104 may classify the biofeedback signal as belonging to the user state. Where multiple fuzzy matches are performed, degrees of match for each respective fuzzy set may be computed and aggregated through, for instance, addition, averaging, or the like, to determine an overall degree of match.

[0072] Still referring to FIG. 5, in an embodiment, a biofeedback signal may be compared to multiple user state fuzzy sets. For instance, biofeedback signal may be represented by a fuzzy set that is compared to each of the multiple user state fuzzy sets; and a degree of overlap exceeding a threshold between the biofeedback signal fuzzy set and any of the multiple user state fuzzy sets may cause computing device 104 to classify the biofeedback signal as belonging to a user state. For instance, in one embodiment there may be two user state fuzzy sets, representing respectively an attentive state and an inattentive state. Attentive state may have an attentive state fuzzy set; inattentive state may have an inattentive state fuzzy set; and biofeedback signal may have a biofeedback fuzzy set. Computing device 104, for example, may compare a biofeedback fuzzy set with each of attentive state fuzzy set and inattentive state fuzzy set, as described above, and classify a biofeedback signal to either, both, or neither of attentive state or inattentive state.

Machine-learning methods as described throughout may, in a non-limiting example, generate coefficients used in fuzzy set equations as described above, such as without limitation x , c , and σ of a Gaussian set as described above, as outputs of machine-learning methods. Likewise, biofeedback signal may be used indirectly to determine a fuzzy set, as biofeedback fuzzy set may be derived from outputs of one or more machine-learning models that take the biofeedback signal directly or indirectly as inputs.

[0073] Referring now to FIG. 6, a flow diagram of an exemplary method 600 for an adaptive machine learning model selection based on data complexity and user goals is illustrated. Method 600 includes step 605 of generating, using at least a processor, a first model as a function of a first dataset and a first set of analytic goals. This may be implemented, without limitation, as described above with reference to FIGS. 1-5.

[0074] With continued reference to FIG. 6, method 600 includes a step 610 of determining, using at least a processor, a complexity metric of a second dataset. This may be implemented, without limitation, as described above with reference to FIGS. 1-5.

[0075] With continued reference to FIG. 6, method 600 includes a step 615 of generating, using at least a processor, a plurality candidate features ff the second dataset using a feature learning algorithm. This may be implemented, without limitation, as described above with reference to FIGS. 1-5.

[0076] With continued reference to FIG. 6, method 600 includes a step 620 of generating, using at least a processor, at least a second model using the plurality of candidate features. This may be implemented, without limitation, as described above with reference to FIGS. 1-5.

[0077] With continued reference to FIG. 6, method 600 includes a step 625 of identifying, using at least a processor, a second complexity gap as a function of the at least a second model using a third dataset. This may be implemented, without limitation, as described above with reference to FIGS. 1-5.

[0078] With continued reference to FIG. 6, method 600 includes a step 630 of selecting, using at least a processor, the at least a second model as a function of the second complexity gap. This may be implemented, without limitation, as described above with reference to FIGS. 1-5.

[0079] It is to be noted that any one or more of the aspects and embodiments described herein may be conveniently implemented using one or more machines (e.g., one or more computing devices that are utilized as a user computing device for an electronic document, one or more server devices, such as a document server, etc.) programmed according to the teachings of the present specification, as will be apparent to those of ordinary skill in the computer art. Appropriate software coding can readily be prepared by skilled programmers based on the teachings of the present disclosure, as will be apparent to those of ordinary skill in the software art. Aspects and implementations discussed above employing software and/or software modules may also include appropriate hardware for assisting in the implementation of the machine executable instructions of the software and/or software module.

[0080] Such software may be a computer program product that employs a machine-readable storage medium. A machine-readable storage medium may be any medium that is capable of storing and/or encoding a sequence of instruc-

tions for execution by a machine (e.g., a computing device) and that causes the machine to perform any one of the methodologies and/or embodiments described herein. Examples of a machine-readable storage medium include, but are not limited to, a magnetic disk, an optical disc (e.g., CD, CD-R, DVD, DVD-R, etc.), a magneto-optical disk, a read-only memory “ROM” device, a random access memory “RAM” device, a magnetic card, an optical card, a solid-state memory device, an EPROM, an EEPROM, and any combinations thereof. A machine-readable medium, as used herein, is intended to include a single medium as well as a collection of physically separate media, such as, for example, a collection of compact discs or one or more hard disk drives in combination with a computer memory. As used herein, a machine-readable storage medium does not include transitory forms of signal transmission.

[0081] Such software may also include information (e.g., data) carried as a data signal on a data carrier, such as a carrier wave. For example, machine-executable information may be included as a data-carrying signal embodied in a data carrier in which the signal encodes a sequence of instruction, or portion thereof, for execution by a machine (e.g., a computing device) and any related information (e.g., data structures and data) that causes the machine to perform any one of the methodologies and/or embodiments described herein.

[0082] Examples of a computing device include, but are not limited to, an electronic book reading device, a computer workstation, a terminal computer, a server computer, a handheld device (e.g., a tablet computer, a smartphone, etc.), a web appliance, a network router, a network switch, a network bridge, any machine capable of executing a sequence of instructions that specify an action to be taken by that machine, and any combinations thereof. In one example, a computing device may include and/or be included in a kiosk.

[0083] FIG. 7 shows a diagrammatic representation of one embodiment of a computing device in the exemplary form of a computer system 700 within which a set of instructions for causing a control system to perform any one or more of the aspects and/or methodologies of the present disclosure may be executed. It is also contemplated that multiple computing devices may be utilized to implement a specially configured set of instructions for causing one or more of the devices to perform any one or more of the aspects and/or methodologies of the present disclosure. Computer system 700 includes a processor 704 and a memory 708 that communicate with each other, and with other components, via a bus 712. Bus 712 may include any of several types of bus structures including, but not limited to, a memory bus, a memory controller, a peripheral bus, a local bus, and any combinations thereof, using any of a variety of bus architectures.

[0084] Processor 704 may include any suitable processor, such as without limitation a processor incorporating logical circuitry for performing arithmetic and logical operations, such as an arithmetic and logic unit (ALU), which may be regulated with a state machine and directed by operational inputs from memory and/or sensors; processor 704 may be organized according to Von Neumann and/or Harvard architecture as a non-limiting example. Processor 704 may include, incorporate, and/or be incorporated in, without limitation, a microcontroller, microprocessor, digital signal processor (DSP), Field Programmable Gate Array (FPGA),

Complex Programmable Logic Device (CPLD), Graphical Processing Unit (GPU), general purpose GPU, Tensor Processing Unit (TPU), analog or mixed signal processor, Trusted Platform Module (TPM), a floating point unit (FPU), system on module (SOM), and/or system on a chip (SoC).

[0085] Memory **708** may include various components (e.g., machine-readable media) including, but not limited to, a random-access memory component, a read only component, and any combinations thereof. In one example, a basic input/output system **716** (BIOS), including basic routines that help to transfer information between elements within computer system **700**, such as during start-up, may be stored in memory **708**. Memory **708** may also include (e.g., stored on one or more machine-readable media) instructions (e.g., software) **720** embodying any one or more of the aspects and/or methodologies of the present disclosure. In another example, memory **708** may further include any number of program modules including, but not limited to, an operating system, one or more application programs, other program modules, program data, and any combinations thereof.

[0086] Computer system **700** may also include a storage device **724**. Examples of a storage device (e.g., storage device **724**) include, but are not limited to, a hard disk drive, a magnetic disk drive, an optical disc drive in combination with an optical medium, a solid-state memory device, and any combinations thereof. Storage device **724** may be connected to bus **712** by an appropriate interface (not shown). Example interfaces include, but are not limited to, SCSI, advanced technology attachment (ATA), serial ATA, universal serial bus (USB), IEEE 1394 (FIREWIRE), and any combinations thereof. In one example, storage device **724** (or one or more components thereof) may be removably interfaced with computer system **700** (e.g., via an external port connector (not shown)). Particularly, storage device **724** and an associated machine-readable medium **728** may provide nonvolatile and/or volatile storage of machine-readable instructions, data structures, program modules, and/or other data for computer system **700**. In one example, software **720** may reside, completely or partially, within machine-readable medium **728**. In another example, software **720** may reside, completely or partially, within processor **704**.

[0087] Computer system **700** may also include an input device **732**. In one example, a user of computer system **700** may enter commands and/or other information into computer system **700** via input device **732**. Examples of an input device **732** include, but are not limited to, an alpha-numeric input device (e.g., a keyboard), a pointing device, a joystick, a gamepad, an audio input device (e.g., a microphone, a voice response system, etc.), a cursor control device (e.g., a mouse), a touchpad, an optical scanner, a video capture device (e.g., a still camera, a video camera), a touchscreen, and any combinations thereof. Input device **732** may be interfaced to bus **712** via any of a variety of interfaces (not shown) including, but not limited to, a serial interface, a parallel interface, a game port, a USB interface, a FIREWIRE interface, a direct interface to bus **712**, and any combinations thereof. Input device **732** may include a touch screen interface that may be a part of or separate from display **736**, discussed further below. Input device **732** may be utilized as a user selection device for selecting one or more graphical representations in a graphical interface as described above.

[0088] A user may also input commands and/or other information to computer system **700** via storage device **724** (e.g., a removable disk drive, a flash drive, etc.) and/or network interface device **740**. A network interface device, such as network interface device **740**, may be utilized for connecting computer system **700** to one or more of a variety of networks, such as network **744**, and one or more remote devices **748** connected thereto. Examples of a network interface device include, but are not limited to, a network interface card (e.g., a mobile network interface card, a LAN card), a modem, and any combination thereof. Examples of a network include, but are not limited to, a wide area network (e.g., the Internet, an enterprise network), a local area network (e.g., a network associated with an office, a building, a campus or other relatively small geographic space), a telephone network, a data network associated with a telephone/voice provider (e.g., a mobile communications provider data and/or voice network), a direct connection between two computing devices, and any combinations thereof. A network, such as network **744**, may employ a wired and/or a wireless mode of communication. In general, any network topology may be used. Information (e.g., data, software **720**, etc.) may be communicated to and/or from computer system **700** via network interface device **740**.

[0089] Computer system **700** may further include a video display adapter **752** for communicating a displayable image to a display device, such as display device **736**. Examples of a display device include, but are not limited to, a liquid crystal display (LCD), a cathode ray tube (CRT), a plasma display, a light emitting diode (LED) display, and any combinations thereof. Display adapter **752** and display device **736** may be utilized in combination with processor **704** to provide graphical representations of aspects of the present disclosure. In addition to a display device, computer system **700** may include one or more other peripheral output devices including, but not limited to, an audio speaker, a printer, and any combinations thereof. Such peripheral output devices may be connected to bus **712** via a peripheral interface **756**. Examples of a peripheral interface include, but are not limited to, a serial port, a USB connection, a FIREWIRE connection, a parallel connection, and any combinations thereof.

[0090] The foregoing has been a detailed description of illustrative embodiments of the invention. Various modifications and additions can be made without departing from the spirit and scope of this invention. Features of each of the various embodiments described above may be combined with features of other described embodiments as appropriate in order to provide a multiplicity of feature combinations in associated new embodiments. Furthermore, while the foregoing describes a number of separate embodiments, what has been described herein is merely illustrative of the application of the principles of the present invention. Additionally, although particular methods herein may be illustrated and/or described as being performed in a specific order, the ordering is highly variable within ordinary skill to achieve methods, systems, and software according to the present disclosure. Accordingly, this description is meant to be taken only by way of example, and not to otherwise limit the scope of this invention.

[0091] Exemplary embodiments have been disclosed above and illustrated in the accompanying drawings. It will be understood by those skilled in the art that various changes, omissions and additions may be made to that which

is specifically disclosed herein without departing from the spirit and scope of the present invention.

What is claimed is:

1. An apparatus for an adaptive machine learning model selection based on data complexity and user goals, wherein the apparatus comprises:

at least a processor; and

a memory communicatively connected to the at least a processor, wherein the memory containing instructions configuring the at least a processor to:

generate a first model as a function of a first dataset and a first set of analytic goals;

determine a complexity metric of a second dataset;

generate a plurality candidate features of the second dataset using a feature learning algorithm;

generate at least a second model using the plurality of candidate features;

identify a second complexity gap as a function of the at least a second model using a third dataset; and

select the at least a second model as a function of the second complexity gap.

2. The apparatus of claim 1, wherein the first model comprises a machine learning model, wherein the machine learning model is trained on the first dataset.

3. The apparatus of claim 1, wherein the first model further comprises a regression algorithm, wherein the regression algorithm is trained to predict sales based on historical data.

4. The apparatus of claim 1, wherein the complexity metric is determined based on a statistical analysis of the second dataset.

5. The apparatus of claim 1, wherein the second dataset comprises a plurality of data profiles, wherein the plurality of data profiles comprises one or more attributes.

6. The apparatus of claim 1, wherein a predetermined threshold is configured to select from a group, wherein the group comprises a number, a set of numbers, a vector of numbers, a fuzzy set, a matching classifier label, and a centroid derived from K-means clustering.

7. The apparatus of claim 6, wherein the at least a processor is further configured to dynamically update the predetermined threshold.

8. The apparatus of claim 1, wherein the processor is further configured to derive a performance score, wherein the performance score is associated with a plurality of parameters.

9. The apparatus of claim 1, wherein the complexity metric is compared to a predetermined threshold, wherein a level of complexity comprises a scale of threshold.

10. The apparatus of claim 1, wherein a first complexity gap is identified as a function of a comparison and the first model.

11. A method for an adaptive machine learning model selection based on data complexity and user goals, the method comprising:

generating a first model as a function of a first dataset and a first set of analytic goals;

determining a complexity metric of a second dataset;

generating a plurality candidate features of the second dataset using a feature learning algorithm;

generating at least a second model using the plurality of candidate features;

identifying a second complexity gap as a function of the at least a second model using a third dataset; and

selecting the at least a second model as a function of the second complexity gap.

12. The method of claim 11, wherein the first model comprises a machine learning model, wherein the machine learning model is trained on the first dataset.

13. The method of claim 11, wherein the first model further comprises a regression algorithm, wherein the regression algorithm is trained to predict sales based on historical data.

14. The method of claim 11, wherein the complexity metric is determined based on a statistical analysis of the second dataset.

15. The method of claim 11, wherein the second dataset comprises a plurality of data profiles, wherein the plurality of data profiles comprises one or more attributes.

16. The method of claim 11, wherein a predetermined threshold is configured to select from a group, wherein the group comprises a number, a set of numbers, a vector of numbers, a fuzzy set, a matching classifier label, and a centroid derived from K-means clustering.

17. The method of claim 16, wherein the predetermined threshold is dynamically updated.

18. The method of claim 11, wherein a performance score is derived, wherein the performance score is associated with a plurality of parameters.

19. The method of claim 11, wherein the complexity metric is compared to a predetermined threshold, wherein a level of complexity comprises a scale of threshold.

20. The method of claim 11, wherein a first complexity gap is identified as a function of a comparison and the first model.

* * * * *